# A Context-Aware Recommendation System for Household Service Providers Using Ensemble Learning in Urban Environments

Student Name: Mbithi, Keith Austine
Student No: 164003

Supervisor: Dr. Deperias Kerre

An Informatics and Computer Science Project Documentation Submitted to the School of Computing and Engineering Sciences in partial fulfillment of the requirements for the award of a Degree in Bachelor of Science in Informatics and Computer Science

**School of Computing and Engineering Sciences**
**Strathmore University**
**Nairobi, Kenya**

**October 2026**

## Declaration and Approval

I declare that this project documentation has not been presented to Strathmore University or any other institution for the award of a Bachelor of Science in Informatics and Computer Science degree, or any other academic qualification. To the best of my knowledge, this document does not contain material previously published or written by another individual, except where proper acknowledgment and references have been provided within the document.

Student Admission Number: 164003

Sign: _____

Date: _____

Supervisor's Signature:

Sign: _____

Date:

## Acknowledgment

I would like to express my sincere gratitude to my supervisor, lecturers, family, and friends for their guidance, encouragement, and support throughout the development of this project. Their assistance and motivation have played an important role in the completion of this work. I also thank Strathmore University for providing a conducive environment and resources that supported this research.

# Abstract

Rapid urbanization in Nairobi continues to increase the demand for household services such as plumbing, electrical repairs, and cleaning. However, existing service-matching platforms primarily rely on distance and user ratings, often overlooking traffic congestion and varying road conditions, resulting in unreliable provider arrivals and reduced user trust.

This study developed a context-aware recommendation system that predicts the arrival reliability of household service providers in Nairobi from environmental and provider-related factors. Because no real booking data were available, a synthetic dataset of 3,000 bookings, calibrated to documented Nairobi traffic patterns and road quality, was used to represent the realities of urban service delivery.

An XGBoost ensemble regression pipeline was trained on five inputs: distance, the congestion multiplier of the provider's corridor, time slot, the road quality of the provider's area and the provider's completion rate. The data were split 80/20 and the hyperparameters were selected by five-fold grid search on the training portion. On the 600 held-out records the model achieved an RMSE of 0.1417, an MAE of 0.1160 and an $R^2$ of 0.4717, against RMSE values of 0.1872 for a distance-only baseline and 0.1936 for a rating-only baseline. Congestion carried about half of the model's importance and distance less than a tenth.

The model was integrated into a working web prototype built with Next.js, FastAPI and PostgreSQL with PostGIS. The prototype ranks available providers, explains each score and supports booking, rescheduling and reviews, and automated backend tests passed. The results hold for simulated data only: the system uses stored traffic patterns and not live traffic, authentication is not verified on the server, and no study with users has been carried out. Validation on real bookings is the main step needed before deployment.



***Keywords: context-aware recommendation, ensemble learning, XGBoost, arrival reliability, household service providers, Nairobi***

# Table of Contents

Declaration and Approval ..... 2
Acknowledgment ..... 3
Abstract ..... 4
Table of Contents ..... 5
List of Figures ..... 9
List of Tables ..... 9
Chapter One: Introduction ..... 1
1.2 Problem Statement ..... 2
1.3 General Objective ..... 2
1.3.1 Specific Objectives ..... 2
1.3.2 Research Questions ..... 3
1.4 Justification ..... 3
1.5 Scope and Limitations ..... 5
1.5.1 Scope of the Project ..... 5
1.5.2 Limitations in the Project ..... 5
1.5.3 Delimitations in the Project ..... 7
Chapter Two: Literature Review ..... 8
2.1 Introduction ..... 8
2.2 Challenges Faced by Users of Service-Matching Platforms ..... 8
2.3 Related Applications ..... 9
2.3.1 Mama Fua App ..... 10
2.3.2 Fundis App ..... 11
2.3.3 Kazi Konekt ..... 12
2.3.4 Context-Aware Recommender Systems ..... 13
2.3.5 XGBoost for Travel Time Prediction ..... 14
Chapter Three: Development Methodology ..... 17
3.1 Introduction ..... 17
3.2 Research Paradigm ..... 17

3.2.1 Data Acquisition ..... 17
3.2.2 Data Preprocessing ..... 18
3.2.3 Model Training ..... 18
3.2.4 Model Validation & Testing ..... 19
3.3 Software Development Methodology ..... 20
3.3.1 Requirements Elicitation ..... 21
3.3.2 System Design ..... 21
3.3.3 Prototyping ..... 21
3.3.4 User Evaluation of Prototype ..... 21
3.3.5 Refinement & Integration ..... 21
3.3.6 Coding & Development ..... 22
3.3.7 Testing & Deployment ..... 22
3.4 System Analysis and Design ..... 22
3.4.1 Use Case Diagram ..... 22
3.4.2 Class Diagram ..... 24
3.4.3 Entity Relationship Diagram (ERD) ..... 24
3.4.4 Database Schema ..... 24
3.4.5 System Architecture ..... 24
3.4.6 Wireframes ..... 25
3.5 System Development Tools and Techniques ..... 25
3.5.1 Python ..... 25
3.5.2 Next.js ..... 25
3.5.3 PostgreSQL and PostGIS ..... 26
3.5.4 XGBoost Ensemble Learning ..... 26
3.5.5 Feature Engineering ..... 26
3.5.6 Git and GitHub ..... 26
3.6 System Deliverables ..... 26
3.6.1 System Documentation ..... 27
3.6.2 Authentication Module ..... 28

3.6.3 Context-Aware Recommendation Engine ..... 28
3.6.4 Provider Booking and Ranking Interface ..... 28
3.6.5 Geospatial Database Module ..... 28
Chapter Four: System Analysis & Design ..... 29
4.1 Introduction ..... 29
4.2 System Requirements ..... 29
4.2.1 Functional Requirements ..... 30
4.2.2 Non-Functional Requirements ..... 31
4.2.3 Data Requirements ..... 33
4.2.4 Interface and Integration Requirements ..... 33
4.3 System Analysis Diagrams ..... 34
4.3.1 Use Case Diagram ..... 34
4.3.2 Sequence Diagram ..... 35
4.4 System Design Diagrams ..... 36
4.4.1 System Architecture ..... 36
4.4.2 Class Diagram ..... 37
4.4.3 Entity Relationship Diagram ..... 37
4.4.4 Logical Database Schema ..... 38
4.4.5 Wireframes ..... 39
4.5 Requirements Traceability ..... 41
Chapter Five: System Implementation, Testing and Evaluation ..... 41
5.1 Introduction ..... 42
5.2 Implementation Environment ..... 42
5.2.1 Hardware and Deployment Environment ..... 42
5.2.2 Software, Services and Configuration ..... 43
5.2.3 Deployment and Reproducibility ..... 44
5.3 System Implementation ..... 45
5.3.1 Database and Data Implementation ..... 45
5.3.2 Dataset Description ..... 46

5.3.3 Data Preparation and Partitioning ..... 47
5.3.4 Model Training and Selection ..... 48
5.3.5 Recommendation Service ..... 50
5.4 Testing and Evaluation ..... 51
5.4.1 Test Strategy and Acceptance Criteria ..... 51
5.4.2 Test Cases, Results, Defects and Retesting ..... 52
5.4.3 Model, Performance and Interface Evaluation ..... 55
5.4.4 Interpretation of the Test Results ..... 56
5.5 Version Control and Implementation Evidence ..... 56
5.6 Author Oversight and Use of AI-Assisted Tools ..... 57
5.7 Evaluation Results, Discussion and Limitations ..... 58
5.7.1 Results by Objective ..... 58
5.7.2 Discussion and Error Analysis ..... 59
5.7.3 Requirement Coverage ..... 60
5.7.4 Limitations ..... 61
References ..... 62
Appendix A1: Time Schedule ..... 65
Appendix A2: System Screenshots ..... 66

# List of Figures

Figure 2.3.1 Mama Fua Booking App
Figure 2.3.2 Fundis Service Booking App
Figure 2.3.3 Kazi Konnekt Dashboard
Figure 2.5.4 Conceptual Framework
Figure 3.3.5 Agile-Scrum Framework
Figure 4.3.1 Use Case Diagram
Figure 4.3.2 Sequence Diagram: Booking Recommendation
Figure 4.4.1 System Architecture Diagram
Figure 4.4.2 Class Diagram
Figure 4.4.3 Entity Relationship Diagram
Figure 4.4.4 Logical Database Schema
Figure 4.4.5 Wireframes: landing page (left) and service request form (right)
Figure 4.4.6 Wireframes: ranked results (left) and provider detail with explanation (right)
Figure 4.4.7 Wireframes: booking confirmation (left) and provider registration (right)
Figure 4.4.8 Wireframes: provider availability (left) and provider dashboard (right)
Figure 4.4.9 Wireframes: mobile layout of the request, results and provider detail screens
Figure 5.3.1 Current database row counts from PostgreSQL
Figure 5.3.2 Actual training dataset preview: model features and target
Figure 5.3.4 Predicted against actual reliability score on the test set
Figure 5.3.5 Recommendation results in the working client interface
Figure 5.4.1 Latency of POST /recommend on the development workstation (200 requests)
Figure A2.1 Landing page
Figure A2.2 Service request form
Figure A2.3 Ranked provider results
Figure A2.4 Booking request sent
Figure A2.5 Client booking list
Figure A2.6 Provider availability
Figure A2.7 Administrator dashboard
Figure A2.8 Provider identity verification

# List of Tables

Table 4.1 Functional requirements
Table 4.2 Non-functional requirements
Table 4.3 Data requirements
Table 4.4 Interface and integration requirements
Table 4.5 Application tables of the logical schema
Table 4.6 Traceability from objectives to requirements, design and verification
Table 5.1 Implementation environment
Table 5.2 Software, libraries and services
Table 5.3 Steps to rebuild the system from the repository
Table 5.4 Summary of the training dataset
Table 5.5 Model comparison on the 600-record test set
Table 5.6 Relative importance of the model features (gain)
Table 5.7 Test levels, tools and acceptance criteria
Table 5.8 Automated backend tests by area
Table 5.9 Selected negative and boundary test cases
Table 5.10 Defects found, corrections and retests
Table 5.11 Reproducibility artefacts in the repository
Table 5.12 Results by research objective
Table 5.13 Test-set error by condition (actual minus predicted score)
Table 5.14 Status of the non-functional requirements
Table 5.15 Limitations of the study

# Chapter One: Introduction

## 1.1 Background

The rapid growth of urban populations has increased the demand for on-demand home and facility services, including electrical repairs, plumbing, and general technical maintenance. In Nairobi, informal service economies provides most of these types of services. Customers look to their social network (word of mouth) physical notice boards, or personal referral to find these service providers; however, with the emergence of digital platforms for matching services with digital providers both globally and locally, the hope is to bring formality to the service profession and help users navigate through the discovery, comparison, and booking process for service providers using web and mobile based interfaces.

Despite this digital shift, the quality and reliability of service delivery remains inconsistent. Users continue to experience delays, no-shows, and substandard work even when engaging providers with strong ratings and competitive pricing. This indicates that there is a structural gap in how the current platforms are analysing and ranking service providers; they evaluate providers in isolation without assessing the dynamic environmental and/or contextual conditions to which a provider must operate within so as to successfully deliver on their promises when providing a service. For example, service providers' ability to successfully complete a job and reach a client can be affected by factors such as road conditions, flooding, traffic congestion, and localised safety conditions.(Ricci et al., 2015).

Over time, recommendation systems in e-commerce and media streaming have evolved significantly, transitioning from static, distance-based matching to intelligent, context-aware recommendation models. These advanced theoretical models dynamically assess the situational conditions under which a service is delivered. However, these improved theoretical methods have not been effectively applied to the informal service sector, especially in urban regions of developing countries where there are large variations in the quality of infrastructure. This project bridges that gap by applying context-aware theories to a machine learning algorithm, providing more reliable recommendations by assessing contextual risks drawn from stored time-slot and corridor traffic patterns.

## 1.2 Problem Statement

Existing service-matching platforms recommend providers based primarily on proximity,

availability, and historical user ratings; a limited set of static parameters that do not account for the real-world conditions that determine whether a provider can actually deliver as expected. In Nairobi, where traffic congestion varies significantly by time of day, day of week, and road corridor, straight-line distance is a poor predictor of actual travel time. A provider located three kilometres away across a heavily congested route may take significantly longer to arrive than one located six kilometres away on a clear road, yet current platforms treat both providers identically from a proximity standpoint.

This mismatch between algorithmic rankings and real delivery performance leads to frequent delays, missed appointments, and eroded user trust in service platforms. The core problem is the absence of contextual awareness in provider-matching algorithms: none of the Kenyan platforms reviewed in Chapter Two incorporates traffic conditions, time-based travel patterns, or road quality into its recommendation logic, leaving users without a reliable basis for predicting whether a chosen provider will arrive on time and complete the job successfully.

## 1.3 General Objective

The aim of this project is to design and develop a context-aware recommendation system for household service providers in urban Nairobi that uses ensemble machine learning to integrate traffic conditions, time-of-day travel patterns, road quality, and provider performance history into a provider reliability score, enabling more accurate and dependable service provider recommendations than proximity-based approaches.

### 1.3.1 Specific Objectives

i. To identify the key factors that influence the reliability of household service providers in urban Nairobi, including traffic conditions, time of day, road quality, and past provider performance.

ii. To review existing Kenyan service-matching platforms and ML-based recommendation systems and identify gaps in contextual awareness and travel-time prediction.

iii. To develop a machine learning model that combines travel and provider-related factors to predict provider arrival reliability.

iv. To evaluate the model's performance against distance-based and rating-based recommendation approaches using RMSE and MAE metrics.

### 1.3.2 Research Questions

i. What contextual factors including traffic congestion, time-of-day patterns, road quality, and provider history most significantly affect the reliability of household service provider delivery in urban Nairobi?
ii. What limitations exist in current Kenyan service-matching platforms and machine learning-based recommendation systems with respect to contextual awareness ?
iii. How can XGBoost ensemble machine learning integrate contextual traffic features and provider performance metrics to generate a reliable provider arrival prediction score ?
iv. How effective is the proposed context-aware model in predicting provider arrival reliability relative to distance-only and rating-only baseline models?

## 1.4 Justification

There are three grounds that justify the development of this system: improved user experience, economic efficiency, and technical contribution. From a user perspective, the persistent gap between expected and actual service delivery represents a significant source of frustration, particularly for households and small businesses that depend on timely technical services. A system that more accurately predicts provider arrival reliability is expected to reduce the incidence of missed appointments, wasted waiting time, and the cost of rescheduling. These benefits are expected and were not measured in this study.

Economically, improved provider matching is expected to benefit all parties. Reliable providers gain more consistent work opportunities; platforms reduce user churn caused by poor delivery experiences; and the informal artisan economy can become more trustworthy and formalised.

From a technical standpoint, while Context-Aware Recommender Systems (CARS) are theoretically well-established in e-commerce, their application to the urban household service domain specifically integrating traffic-based travel time modeling and ensemble learning appeared largely unexplored in the Kenyan platforms and literature reviewed. This project contributes a theoretically grounded approach, utilizing Gradient Boosting theory via XGBoost to predict provider arrival reliability as a function of

contextual traffic features, time-of-day patterns, and historical provider performance.

## 1.5 Scope and Limitations

### 1.5.1 Scope of the Project

The system was delivered as a responsive web application that works on desktop and mobile browsers and can be installed on a phone. It covers five categories of household service providers in the Nairobi metropolitan area: electricians, plumbers, cleaners, technicians and carpenters. The functionality includes registration and login for clients and providers, service requests with a named area or an exact map location, context-aware provider ranking using the trained XGBoost model with an explanation for each score, availability management for providers, booking with status changes, cancellation, rescheduling, reviews, notifications and a limited administrator console.

The machine learning component covers data preparation, feature engineering, model training with XGBoost ensemble learning, and evaluation against distance-only and rating-only baseline models. The trained model uses five inputs, one for each conceptual pillar: the straight-line distance between client and provider, the congestion multiplier of the provider's corridor for the time slot, the time slot itself, the road quality of the provider's area and the provider's completion rate. Other attributes are used around the model and are not model inputs: the estimated travel time is calculated for display and for the explanation, the job date sets the day type used to look up traffic patterns, and the provider's rating is used by the rating-only baseline and as an optional filter. A PostgreSQL database with the PostGIS extension manages provider and spatial data. Provider onboarding and profile management are included within the system scope.

### 1.5.2 Limitations in the Project

The system's training data was synthetically generated from realistic Nairobi traffic patterns and provider behaviour distributions. While the data is grounded in documented Nairobi road conditions and congestion characteristics, it does not capture the full complexity of real-world variability. Model performance may differ once the system is run against live data, and retraining on real booking outcomes would be necessary over time to maintain predictive quality.

The system does not integrate real-time traffic APIs in its initial implementation; traffic features are based on time-slot and corridor patterns derived from historical averages. Real-time API integration is identified as a priority extension. The system does not cover every service provider category, nor does it extend beyond the Nairobi metropolitan area. Payment processing and live provider GPS tracking fall outside the scope of this project. Authentication is not verified on the

server, so the prototype does not provide production-grade access control.

### 1.5.3 Delimitations in the Project

The project deliberately excludes mobile application development (iOS/Android), focusing solely on a responsive web-based interface to manage development scope and ensure cross-device accessibility without platform-specific builds. Flood zone analysis, crime risk integration, and real-time weather data while relevant to a broader risk-aware recommendation system are beyond the scope of this project and are identified as future work extensions. The system does not attempt to resolve disputes between users and providers, manage in-person service quality inspection, or integrate with third-party payment gateways. These delimitations allow the project to maintain a focused, technically rigorous implementation within the timeframe provided for the project while delivering a genuinely novel and functional recommendation system.

# Chapter Two: Literature Review

## 2.1 Introduction

This chapter reviews existing literature and related systems relevant to the development of a context-aware recommendation system for household service providers in urban Kenya. The review is structured around four areas: the challenges that define the urban service delivery problem in the Kenyan context, with particular focus on traffic congestion and travel time variability; a critical analysis of three existing Kenyan service-matching platforms and how each addresses or fails to address the contextual gap; the specific limitations and gaps that the proposed system is designed to overcome; and a conceptual framework that maps the system's processing pipeline from raw inputs to ranked provider recommendations. The literature and platform analysis presented in this chapter directly inform the design and architectural decisions described in subsequent chapters.

## 2.2 Challenges Faced by Users of Service-Matching Platforms

The demand for on-demand household services in urban Kenya has grown substantially alongside rapid urbanisation, with Nairobi's population exceeding five million and continuing to expand into densely populated peri-urban settlements such as Mathare, Kibera, and Ruiru (Kenya National Bureau of Statistics, 2019). This growth has created a large and underserved market for skilled tradespeople electricians, plumbers, cleaners, and general technicians many of whom operate informally, without consistent access to clients or reliable income. The emergence of digital service-matching platforms in Kenya over the past decade has sought to formalise this market, but the platforms that have developed face a structural challenge: matching algorithms designed for well-connected, infrastructure-stable environments do not translate effectively to Nairobi's complex urban geography.

A fundamental challenge in urban service delivery is the extreme variability of actual travel times across the road network. The majority of service providers commute by public transit or motorcycle, making their journey times highly sensitive to both road surface conditions and prevailing traffic levels. Existing platforms fail to address this because they rely on the naive theoretical approach of using straight-line (Haversine) distance as a proxy for travel time. Theoretical studies in transportation modeling demonstrate that straight-line distance produces significant errors in dense urban environments where road network topology, congestion, and

surface quality dominate actual journey duration. To overcome this structural challenge, travel time must be modeled as a function of distance, free-flow speed, and a contextual congestion multiplier that varies by route corridor and time of day.

Traffic congestion compounds this challenge further. Nairobi is consistently ranked among the most congested cities in Africa, with average rush-hour commute times up to three times longer than off-peak times along major corridors such as Mombasa Road, Thika Road, and Ngong Road (TomTom International BV, 2025). A straight-line or basic GPS distance calculation the standard used by most Kenyan service platforms is therefore a deeply unreliable proxy for actual travel time and provider reachability. A provider located two kilometres away across a congested junction may take longer to arrive than one located five kilometres away on a clear route, yet current platforms rank these providers identically from a distance perspective.

Together, road quality variability and traffic congestion patterns define a category of contextual challenge that is specific to the Nairobi urban environment and that existing recommendation systems are structurally unable to address. Because platforms rely on static proximity as a matching criterion, two providers at equal distances from a client are treated as equivalent regardless of whether one faces a congested corridor and the other a clear bypass. A system that can model these differences using time-of-day traffic patterns, road quality data, and historical provider performance can produce materially more reliable recommendations than any distance-based approach (Adomavicius et al., 2011).

## 2.3 Related Applications

Before analyzing specific local applications, it is necessary to establish the theoretical machine learning frameworks that govern intelligent provider matching. Traditional recommender systems generate suggestions based solely on user preferences and item attributes. However, Context-Aware Recommender Systems (CARS) theory extends this paradigm by incorporating situational variables such as time, location, and environmental conditions directly into the recommendation function, consistently outperforming context-free models. Furthermore, to process these complex, non-linear contextual variables, Ensemble Learning theory is utilized. Techniques like Gradient Boosting sequentially build decision trees to correct the residual errors of previous models. An optimized implementation of this theory, XGBoost, is particularly effective for structured tabular datasets and

forms the theoretical baseline for modern, intelligent matching. Despite these theoretical advancements, current Kenyan platforms exhibit significant gaps in their implementations.

### 2.3.1 Mama Fua App

Mama Fua is a Kenyan service-matching platform focused on cleaning, laundry, and fumigation services, operating across Nairobi, Mombasa, and Kisumu. The platform is notable for its social mission: it primarily onboards and empowers female workers, partnering with Cleaning School Kenya to deliver standardised skills training before any worker is listed on the platform. Users book services by selecting a service category and providing their location, after which the system matches them with an available, trained provider in their area.

Mama Fua's approach to service quality is human-centred rather than algorithmic: reliability is guaranteed through vetting and training, not through intelligent matching. While this model addresses the competency dimension of provider quality effectively, it creates a significant contextual gap. The platform relies on standard location-based matching and does not dynamically adjust estimated arrival times, provider rankings, or booking recommendations in response to environmental conditions. During Nairobi's heavy rainfall periods, when the ability of a cleaner to commute via public transit is severely compromised, the platform offers no mechanism to flag elevated delivery risk, recommend a closer alternative, or communicate realistic adjusted arrival expectations to the user. The system's quality guarantee is rendered fragile by conditions the algorithm was never designed to consider.

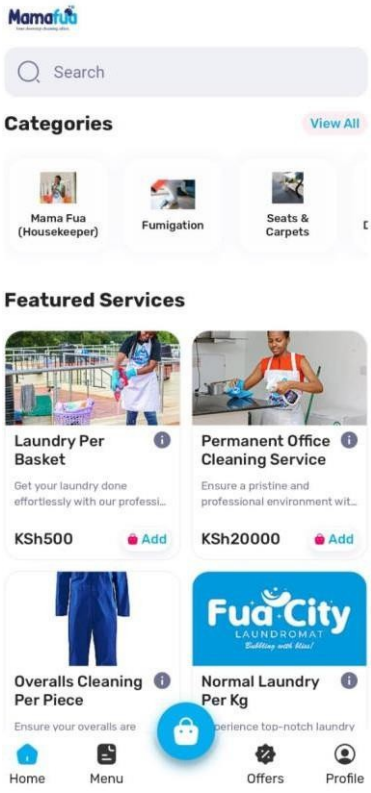



*Figure 2.3.1 Mama Fua Booking App*

### 2.3.2 Fundis App

Fundis App, also referred to as Fundilink, is a broad marketplace connecting users with skilled tradespeople including plumbers, electricians, carpenters, and welders across Kenya. The platform incorporates a two-way review system, allowing both clients and providers to rate each other, and offers digital portfolios where providers can showcase completed work. Fundis also provides a work quality guarantee for a defined period following job completion, introducing a degree of accountability beyond the booking transaction.

Despite these features, the Fundis App's core matching logic is based on straight-line geolocation surfacing the nearest available provider to the user's pin location. This approach, while intuitive, is fundamentally flawed in the context of Nairobi's urban geography. A plumber located three kilometres away across a heavily congested corridor or through an area of poor road infrastructure presents a materially higher operational risk than one located six kilometres away on a clear, accessible route.

The application lacks the spatial intelligence to compute route-based accessibility scores or to

factor road quality and real-time transit times into its ranking. The consequence is that a metric the platform presents as a measure of convenience proximity is in practice an unreliable and sometimes misleading predictor of actual provider reachability. This is the gap that route-aware contextual features, such as corridor congestion and road quality, are designed to close.

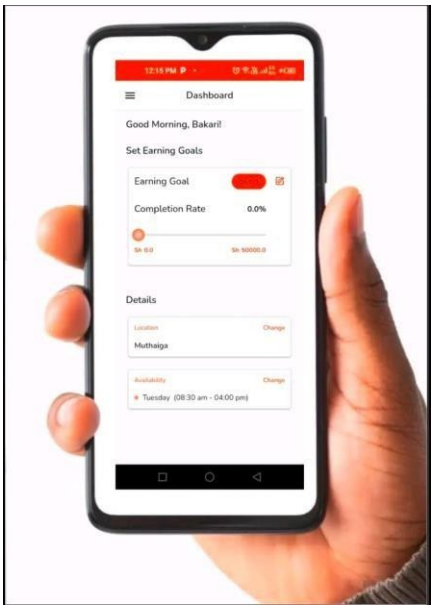



*Figure 2.3.2 Fundis Service Booking App*

### 2.3.3 Kazi Konekt

Kazi Konekt is an on-demand service application built around a dual-interface model comprising a Worker App and a Client App, aimed at formalising Kenya's gig economy. The platform distinguishes itself from competitors through its use of an AI-first intelligent worker dispatch system, which prioritises speed of response and worker availability. Providers are incentivised through a badge and rating system that rewards completed jobs with higher visibility and preferential dispatch, effectively building a dynamic ranking layer on top of availability-based matching.

Kazi Konekt represents the most technically advanced of the Kenyan platforms reviewed, and its AI dispatch layer is a meaningful step toward intelligent provider matching. However, the algorithm is optimised for a single dimension: speed of dispatch based on availability and proximity. It does not account for time-of-day traffic conditions or road quality on the route between a provider and a client. A concrete illustration of this limitation is the scenario where a client in Embakasi requests a plumber at 8:00 AM on a weekday. Kazi Konekt's system would

dispatch the nearest available provider without considering that Mombasa Road at that hour typically operates at a fraction of its free-flow speed, meaning a provider six kilometres away on a less congested route may realistically arrive sooner and more reliably than the nearest provider across the gridlock. The absence of temporal traffic awareness in an otherwise intelligent system means that the platform's speed optimisation can, under predictable daily conditions, produce suboptimal matching outcomes for both client and provider.

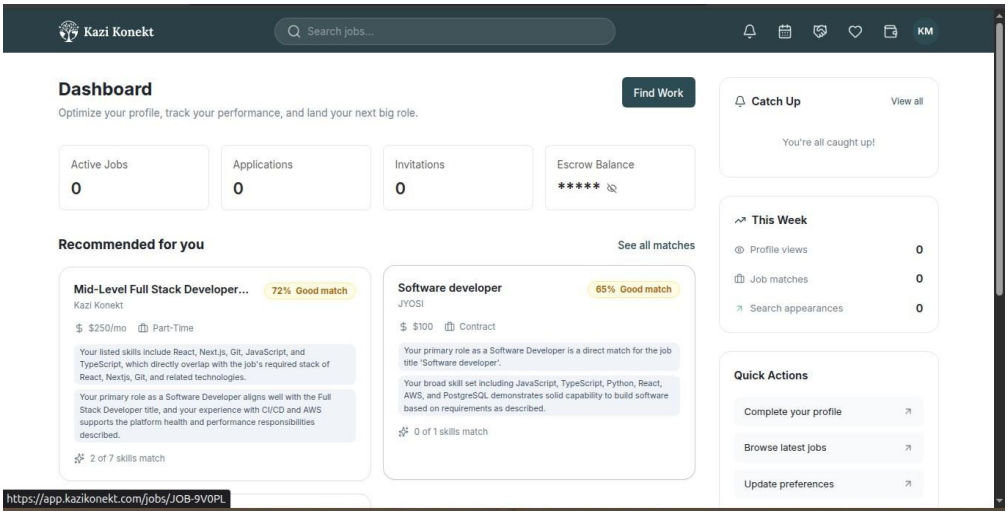



*Figure 2.3.3 Kazi Konnekt Dashboard*

### 2.3.4 Context-Aware Recommender Systems

Adomavicius, Mobasher, Ricci, and Tuzhilin (2011) introduced the concept of Context-Aware Recommender Systems (CARS), which extends traditional recommendation methods by incorporating contextual variables such as time, location, and environmental conditions into the recommendation process. The study demonstrated that considering contextual information enables systems to produce recommendations that are more relevant to users' current situations, overcoming the limitations of static recommendation approaches.

However, the work primarily provides a conceptual framework and discusses several application domains rather than focusing on urban service-provider matching. Furthermore, the study does not incorporate transportation variables such as traffic congestion or road quality, which are critical factors in the Nairobi context. Nevertheless, the theoretical principles established in this work provide the conceptual foundation for the present study.

### 2.3.5 XGBoost for Travel Time Prediction

Huang et al. (2020) investigated the use of tree-based ensemble methods, including XGBoost, for travel time prediction using structured transportation datasets. Their study showed that ensemble learning models effectively capture nonlinear relationships between temporal and spatial variables and outperform simpler methods in prediction accuracy. The research highlighted the suitability of gradient boosting techniques for handling complex interactions among traffic-related features.

Despite its strong predictive performance, the study focused exclusively on estimating travel duration rather than ranking alternative service providers according to their likelihood of successful service delivery. The work therefore predicts trip characteristics rather than provider reliability. Nonetheless, its feature engineering approach and the demonstrated effectiveness of ensemble learning directly motivate the use of XGBoost in the present study.

## 2.4 Gaps in the Existing Applications

A structured comparison of the three platforms reviewed above reveals a consistent and significant pattern of gaps that collectively define the problem this project addresses. The most critical gap, present across all three platforms, is the complete absence of time-of-day traffic awareness in provider matching. None of the reviewed platforms adjusts its recommendations based on when a booking is made relative to known congestion patterns on the likely route between provider and client. Proximity is used as a direct proxy for reachability, and a provider's aggregate rating is used as a proxy for reliability, when neither correlation holds consistently in Nairobi's traffic-variable environment. A provider ranked first at 10:00 AM on a Saturday may be the worst choice at 7:30 AM on a Monday morning along the same corridor, yet current platforms make no such distinction.

The second gap is the absence of contextual travel time modelling. Nairobi's traffic conditions change significantly by time of day, day of week, and road corridor. A booking made at 8:00 AM on a weekday along Mombasa Road involves fundamentally different travel dynamics than one made at 2:00 PM on a Saturday, yet none of the platforms reviewed model this difference. Kazi Konekt comes closest with its AI dispatch system, but even this operates entirely on internal platform data, specifically worker availability and job history, without incorporating any traffic signal or road quality variable into its matching logic.

Third, while Kazi Konekt employs an AI-based dispatch layer, no reviewed platform uses a machine learning model to generate a multi-factor provider reliability score. The distinction matters: dispatch optimisation selects the fastest-to-respond available provider, whereas reliability scoring predicts which provider is most likely to successfully complete a job under current

conditions. The latter requires feature engineering that combines environmental, spatial, and provider-profile data into a single predictive model a capability that XGBoost ensemble learning is well-suited to provide (Chen & Guestrin, 2016).

Finally, none of the reviewed platforms exposes the basis of its matching decisions to users in a transparent or explainable format. Users are presented with a list of providers without understanding why a particular provider was ranked first, what risk factors were or were not considered, or how conditions on the day of booking might affect the recommendation. This opacity undermines informed decision-making and reduces the ability of users to calibrate their expectations appropriately.

## 2.5 Conceptual Framework

The conceptual framework for this project describes how the system's components interact to transform raw inputs into a context-aware, ranked provider recommendation. It is designed to directly address the four gaps identified in Section 2.4 by introducing time-of-day traffic modelling, road quality scoring, ensemble machine learning, and an explainable output layer into a unified processing pipeline.

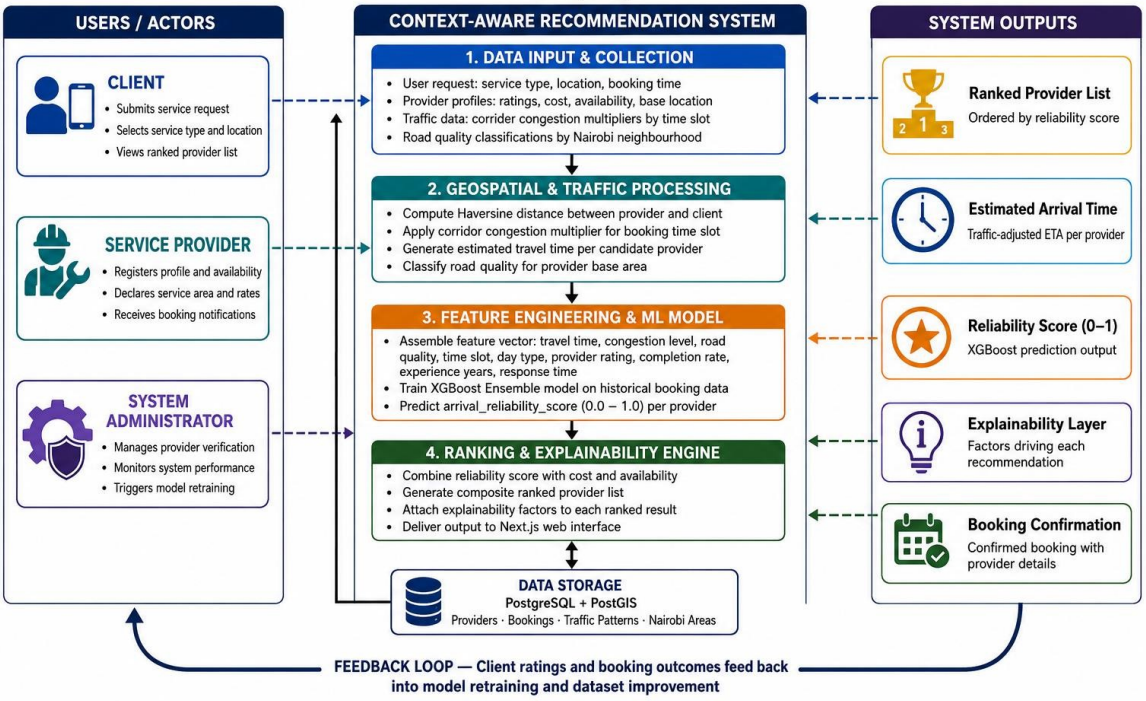
Three categories of input initiate the pipeline. The first is the user request, capturing the service type required, the client's location, and the preferred booking time. The second is the provider database, holding structured provider profiles including historical ratings, declared availability, service specialisations, cost, and geographic base location stored in PostgreSQL. The third is contextual traffic data: pre-computed traffic congestion multipliers by road corridor and time slot, and road quality classifications by area, both stored in the PostgreSQL database and used to estimate realistic travel times between each candidate provider and the client.

These inputs are processed through two analytical stages before reaching the machine learning model. First, travel time estimation combines the provider's distance from the client with the congestion multiplier for the relevant corridor and time slot, producing a realistic estimated travel time for each candidate provider. Simultaneously, feature engineering assembles the five-feature vector for each candidate: distance, the congestion multiplier, the time slot, the road quality of the provider's area and the provider's completion rate. The estimated travel time is shown to the client and used in the explanation, but it is not a model input. The XGBoost ensemble model receives these feature vectors and outputs a reliability prediction score for each

candidate, and the providers are ordered by that score alone. Availability removes candidates before scoring, and the client may narrow the list by hourly rate, rating or verification status, but these filters do not change the scores or the order.

The output is delivered to the user through the web-based interface, built in Next.js, which presents the ranked provider list alongside the key contextual factors that influenced each provider's position. This transparency layer is the system's response to the opacity gap identified in the platform review: users can see not just who is recommended, but whywhether a provider scored well because of a favourable time-of-day traffic profile, a clear route to the client, or a strong historical completion record. Figure 2.5.4 shows the pipeline as originally conceived; in the implemented prototype the model receives only the five features named above (Section 5.3.3).

*Figure 2.5.4 Conceptual Framework*



# Chapter Three: Development Methodology

## 3.1 Introduction

This chapter outlines the methodology and strategies that were employed in the development of the context-aware recommendation system for household service providers. It details the quantitative experimental research paradigm governing the machine learning pipeline, alongside the Agile software development methodology used to build the web-based Interactive Decision Support System (IDSS). Furthermore, the chapter breaks down the system analysis and design components, the specific development tools and techniques utilized, and the system deliverables.

## 3.2 Research Paradigm

This project adopts a quantitative experimental research design grounded in the CRISP-DM framework. The experiment is structured around three model conditions that differ only in the set of independent variables (features) provided to the recommendation algorithm. The dependent variable across all conditions is the arrival_reliability_score a composite metric (0.0–1.0) reflecting predicted provider on-time arrival and job completion probability.

### 3.2.1 Data Acquisition

The synthetic dataset was generated using a rule-based simulation framework implemented in Python. The simulation parameters were derived from documented characteristics of the Nairobi road network. Congestion multipliers and peak-hour time windows were calibrated against TomTom Traffic Index data (TomTom, 2025), which records average rush-hour speeds of 16.7–20.5 km/h and an evening congestion level of 98.8% across the city. Speed reduction factors on major corridors were further validated against Uber Movement data, which confirms speeds on Mombasa Road, Uhuru Highway, and Waiyaki Way run 60–80% below free-flow during peak hours (Shah, 2021). Road quality classifications by neighborhood were grounded in the road access deprivation framework applied to Nairobi by Haer et al. (2025) and surface type data from the Kenya Roads Board's Road Inventory and Conditions Survey (KRB, 2023). The ten modeled road corridors correspond to primary congestion routes identified in the JICA Nairobi Urban Transport Master Plan Study (JICA, 2006) and confirmed by Daily Nation traffic analysis (Daily Nation, 2025). This grounding ensures that while the dataset is synthetic, its distributional properties reflect the empirically documented realities of Nairobi.

### 3.2.2 Data Preprocessing

Before training, the dataset was checked for missing values, and none were found in the model

columns. The two categorical inputs, time_slot and provider_area_road_quality, are one-hot encoded inside the saved pipeline, so the encoder is fitted on training data only. The numeric inputs pass through unchanged, because gradient-boosted trees are unaffected by monotonic scaling. The estimated travel time, which combines the Haversine (straight-line) distance with the congestion multiplier of the corridor, is calculated for display and for the explanation. It is not a model input, because the model receives the distance and the congestion multiplier separately. The columns that are known only after a job ends were excluded to prevent leakage of the outcome into the prediction.

### 3.2.3 Model Training

The experimental design manipulates the independent variables across controlled model conditions. An initial candidate set of about twenty variables covered contextual features (congestion multiplier, estimated travel time, road quality, time slot, day type), provider performance features (rating, completion rate, experience, response time) and spatial distance. It was reduced to five features, one per conceptual pillar, after testing showed that the reduced set matched the performance of the larger one: distance, congestion multiplier, time slot, provider-area road quality and provider completion rate. The dependent variable across all conditions is the arrival_reliability_score, a continuous value from 0.0 to 1.0 that reflects the on-time arrival and job completion of a provider in the synthetic data.

The models were trained under three conditions to isolate the effect of the contextual features. Baseline A – Distance-Only: a linear regression predicts the score from the straight-line distance alone, as a control that represents current platforms. Baseline B – Rating-Only: a linear regression predicts the score from the provider's historical rating alone, as a second control without spatial or contextual data.

**Proposed Model: an XGBoost ensemble model is trained on the five selected features. The data were split 80/20 at random with a fixed seed, and the hyperparameters (learning rate, maximum tree depth and minimum child weight) were selected by five-fold grid search on the training portion only.**

### 3.2.4 Model Validation & Testing

All three conditions are evaluated on the same held-out test dataset. The models are assessed using two primary metrics:

- Root Mean Square Error (RMSE): Measures the average magnitude of prediction errors,

penalizing large variances heavily. This is crucial for safety-relevant predictions like arrival reliability.

- Mean Absolute Error (MAE): Measures the average absolute difference between prediction and actual value, indicating how far off predictions are on the 0–1 reliability scale.

Additionally, five-fold cross-validation was applied to the proposed XGBoost model. The technique splits the dataset into five equal parts, training on four and testing on one, rotating until every record has been tested. It indicates whether the model generalizes across different subsets of the data and helps to detect overfitting. The core experimental hypothesis was that the proposed model would produce lower RMSE and MAE scores than both baseline conditions, which would support the inclusion of contextual traffic features.

## 3.3 Software Development Methodology

The development of the web-based Interactive Decision Support System (IDSS) utilizes the Agile Iterative Prototyping methodology. While the machine learning pipeline is governed by the CRISP-DM framework outlined in Section 3.2, the frontend and API layers require a highly flexible approach. Agile Iterative Prototyping is justified for this project because the requirements for user interface behavior, dynamic provider ranking displays, and the explainability presentation are best refined through repeated evaluation cycles rather than fixed, rigid upfront specifications.

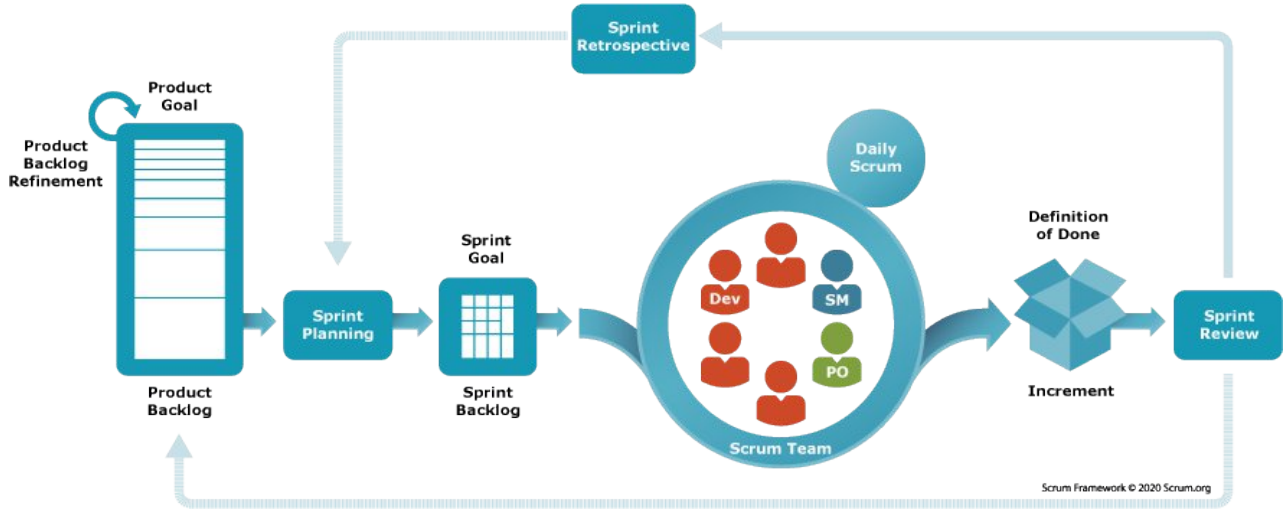



*Figure 3.3.5 Agile-Scrum Framework*

### 3.3.1 Requirements Elicitation

This phase specified the functional and non-functional requirements of the system. The functional requirements covered the service request form, the context-aware provider ranking display and the booking workflow. The non-functional requirements included the response time of a recommendation request, which passes from the Next.js frontend through the FastAPI backend to the XGBoost model. The requirements are listed in Chapter Four.

### 3.3.2 System Design

Following requirements elicitation, the structural blueprints of the system were drafted. This phase covered the software architecture, the PostgreSQL and PostGIS database schema and the Object-Oriented Analysis and Design (OOAD) diagrams, namely the use case, class and entity-relationship diagrams. Low-fidelity wireframes were also drawn to fix the layout of the dashboards and the results page.

### 3.3.3 Prototyping

A minimal Next.js prototype was built from the design specifications. In this iteration the machine learning backend and the spatial database were bypassed, and the prototype was populated with static mock data to validate the layout, the cross-device responsiveness and the navigation from service request to booking confirmation.

### 3.3.4 User Evaluation of Prototype

A usability evaluation of the prototype was planned, with the explainability layer as its main focus: whether users can interpret why a provider received a given reliability score. This evaluation with users was not completed. The informal usability check remains an open task and is recorded as a limitation in Section 5.7.4.

### 3.3.5 Refinement & Integration

The prototype was then refined and integrated. The static mock data was removed and the Next.js frontend was connected to the FastAPI backend.

The PostgreSQL database with the PostGIS extension was integrated to store the spatial data of areas, clients and providers. The Haversine distances are calculated in Python inside the API and not by PostGIS queries. The prediction endpoint feeds the reliability scores into the interface.

### 3.3.6 Coding & Development

This phase covered the programming of the integrated system. The backend API and the machine

learning logic were written in Python, the relational and spatial data were stored in PostgreSQL with the PostGIS extension, and the frontend components, state management and API calls were written with the Next.js React framework. Version control was maintained with Git and GitHub throughout (Section 5.5).

#### 3.3.7 Testing & Deployment

Testing combined several levels. Unit and integration tests of the Python code, the API and the database were written with pytest, and browser tests of the main user flows were written with Playwright. The results are reported in Section 5.4. The frontend components were not unit tested separately. For demonstration, the stack was deployed to free-tier hosted services (Vercel, Render and Neon); this deployment is not a production environment.

## 3.4 System Analysis and Design

System analysis and design form the blueprinting phase of the development lifecycle, translating elicited requirements into structured visual models. This section utilizes standard Object-Oriented Analysis and Design (OOAD) artifacts to map the system's behavior, static structure, and data flow. These models ensure that the complex interactions between the Next.js frontend, the PostgreSQL database, and the Python-based machine learning engine are logically sound prior to full-scale coding.

#### 3.4.1 Use Case Diagram

A use case diagram models the dynamic behaviour of the system by defining the interactions between external actors and the functionality of the system. The actors are the Client, the Service Provider and the System Administrator. The Client requests a service, views ranked recommendations with their explanations, books, cancels, reschedules and reviews; the Provider sets availability and responds to, completes or reschedules bookings; and the Administrator reviews provider profiles, monitors the platform and triggers model retraining. The final diagram is presented and discussed in Section 4.3.1.

#### 3.4.2 Class Diagram

The class diagram maps the static structure of the recommendation pipeline, defining the classes, their attributes and the methods they execute. The classes drawn are the request context, the feature vector, the reference-data store, the trained pipeline and the explanation builder. For example, the pipeline exposes a prediction method that receives the feature vector and returns a

reliability score. The booking, account and notification services are not modelled as classes in this diagram. The final diagram is presented in Section 4.4.2.

### 3.4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram visually represents the data architecture, detailing how distinct database tables relate to one another. This is critical for maintaining data integrity across the synthetic Nairobi dataset. The ERD illustrates the primary and foreign key relationships connecting the clients table, service_providers table, nairobi_areas (holding a centroid point for each area), traffic_patterns, and the central historical_bookings table. This mapping ensures that complex

queries such as fetching a provider's historical performance joined with their base location can be executed efficiently.

### 3.4.4 Database Schema

The database schema provides the logical, tabular breakdown of the ERD, explicitly defining data types, constraints, and index structures. Because the system relies heavily on geographic calculations, the schema heavily features the PostgreSQL implementation enhanced by the PostGIS extension. It details the specific spatial data types (e.g., GEOMETRY(Point, 4326)) used to store client and provider coordinates, alongside standard relational definitions for strings, integers, and the floating-point constraints of the arrival_reliability_score.

### 3.4.5 System Architecture

The system architecture diagram provides a high-level topographical view of the entire technology stack and the directional flow of data. It visually separates the system into three distinct tiers: the Client Tier (Next.js web application running in a browser), the API/Application Tier (handling routing and business logic), and the Data/Processing Tier (the PostgreSQL/PostGIS database and the Python/XGBoost execution environment). This diagram justifies the technological choices by showing how user requests are routed to the machine learning engine and how predictive responses are returned.

### 3.4.6 Wireframes

Wireframes serve as the low-fidelity structural blueprints for the user interface. They dictate the layout of information and interactive elements on the screen without addressing aesthetic design choices like color or typography. The wireframes for this IDSS detail the layout of the user

dashboard, the location and service input forms, and crucially, the results page. The results wireframe demonstrates how the dynamically ranked list of providers is displayed alongside the explainability layer, ensuring the user can easily interpret why a provider was assigned a specific reliability score.

## 3.5 System Development Tools and Techniques

The successful implementation of this context-aware recommendation system relies on a specific stack of modern development tools and data science techniques, chosen to handle both web application deployment and complex algorithmic processing.

### 3.5.1 Python

Python is utilized as the primary programming language for the backend machine learning pipeline. Its extensive ecosystem of data manipulation libraries (such as Pandas and NumPy) makes it the optimal choice for preprocessing the synthetic Nairobi dataset, engineering spatial and temporal features, and training the predictive model.

### 3.5.2 Next.js

Next.js, a robust React framework, is selected for developing the Interactive Decision Support System (IDSS) frontend. It allows for the rapid construction of a responsive, cross-device web application. Next.js was chosen for its server-side rendering capabilities and streamlined API route integration, although in the implemented system the browser calls the FastAPI service directly.

### 3.5.3 PostgreSQL and PostGIS

PostgreSQL 16 serves as the primary relational database management system, selected for its reliability and extensibility. The PostGIS 3.4 extension allows the system to store Nairobi coordinates natively as point geometries (SRID 4326) with spatial indexes. The straight-line (Haversine) distance between a client and a provider is calculated in Python inside the API and not by a PostGIS query.

### 3.5.4 XGBoost Ensemble Learning

eXtreme Gradient Boosting (XGBoost) is the core machine learning technique utilized to generate the provider arrival reliability score. As a decision-tree-based ensemble algorithm, it is highly optimized for structured, tabular data. It was chosen for its execution speed, accuracy, and built-in regularization techniques that prevent model overfitting during the training phase.

### 3.5.5 Feature Engineering

Feature engineering is a data science technique applied during preprocessing that transforms raw data into predictive variables. For this system it assembles the five model inputs: the straight-line distance between client and provider, the congestion multiplier looked up for the corridor and time slot, the time slot, the road quality of the provider's area and the provider's completion rate. A travel-time estimate is also derived from the distance and the congestion multiplier, for display and for the explanation.

### 3.5.6 Git and GitHub

Git is utilized as the version control technique to track iterative changes across both the Next.js codebase and the Python machine learning scripts. GitHub serves as the remote repository hosting the project, facilitating secure backups and providing a structured environment for documenting the development process.

## 3.6 System Deliverables

### 3.6.1 System Documentation

A comprehensive, academically rigorous research report encompassing Chapters One through Six. This documentation details the problem statement, literature review, methodology, implementation procedures, evaluation metrics, and final conclusions drawn from the research.

### 3.6.2 Authentication Module

A user management module integrated into the web application. It enables clients and service providers to register, log in, view their profiles and manage their bookings. Passwords are stored as bcrypt hashes. Identity is not yet verified on the server, so the module is a prototype and not production access control.

### 3.6.3 Context-Aware Recommendation Engine

The core backend processing module housing the trained XGBoost pipeline. It receives a feature vector and returns an arrival reliability score based on Nairobi's spatial and traffic patterns, and it is exposed through the FastAPI service.

### 3.6.4 Provider Booking and Ranking Interface

The user-facing Next.js web application. This deliverable allows users to input their service requirements and displays the algorithmic output as a transparent, ranked list of providers, complete with booking confirmation functionalities.

### 3.6.5 Geospatial Database Module

The structured and populated PostgreSQL database with the PostGIS extension. It holds the synthetic dataset, the reference data and the live application records, and the recommendation engine uses it through the API.

# Chapter Four: System Analysis & Design

## 4.1 Introduction

This chapter presents the analysis and design of the NaiServe context-aware recommendation system. It states what the system must do, the quality conditions it must satisfy, the data it depends on and the interfaces through which its users and components interact. It then communicates the structure and behaviour of the system through a set of analysis and design diagrams.

The chapter is organised as follows. Section 4.2 specifies the functional, non-functional, data and interface requirements. Section 4.3 presents the analysis diagrams, namely the use case diagram and the sequence diagram of the recommendation request. Section 4.4 presents the design diagrams: the system architecture, the class diagram, the entity relationship diagram, the logical database schema and the wireframes. Section 4.5 traces the requirements to the project objectives, the diagrams and the planned verification.

As justified in Section 3.3, the web application and API layers follow Object-Oriented Analysis and Design (OOAD), while the machine learning pipeline follows the CRISP-DM process described in Section 3.2. UML diagrams therefore form the core of the analysis. The data pipeline is represented through the sequence diagram, the class diagram and the logical schema rather than through data-flow diagrams, which belong to structured analysis and were not adopted. The diagrams were prepared at the design stage of the project. Where the implemented prototype later extended or departed from them, the difference is noted beside the diagram and recorded in Chapter 5.

## 4.2 System Requirements

The system requirements identified for this project are presented in the following subsections. Each requirement has a unique identifier, a priority expressed as Must, Should, Could or Won’t (this release), a source and a verification method, so that it can be traced to the objectives in Section 1.3.1 (referred to here as Objectives i to iv) and to the test evidence in Chapter 5. The requirements were framed using the guidance of ISO/IEC/IEEE 29148:2018, and the quality areas in the non-functional requirements follow the product-quality model of ISO/IEC 25010:2023 (ISO/IEC, 2023; ISO/IEC/IEEE, 2018).

### 4.2.1 Functional Requirements

Functional requirements describe what the system must do under specified conditions. They are listed in Table 4.1. Requirements FR-03 to FR-08 describe the recommendation service that is the core contribution of the project, FR-09 to FR-15 describe the booking and account services built around it, and FR-17 records the administrator capability from the use case model.

*Table 4.1 Functional requirements*

| ID | Requirement | Priority | Verification |
|---|---|---|---|
| FR-01 | The system shall allow a visitor to register as a client or a service provider, validate the fields required for the chosen role, and store the password only as a bcrypt hash. | Must | API tests; browser demonstration |
| FR-02 | The system shall authenticate a registered user by email and password and open the dashboard that matches the user's role (client, provider or administrator). | Must | API tests; end-to-end test |
| FR-03 | The system shall accept a service request made of a service type, a job location (a named area or a pin on a map), a time slot and a job date no more than 60 days ahead. | Must | API validation tests; end-to-end test |
| FR-04 | The system shall exclude every provider with no availability record for the requested weekday and time slot before any provider is scored. | Must | Availability tests |
| FR-05 | The system shall predict an arrival reliability score between 0.0 and 1.0 for each eligible provider from five features (distance, congestion multiplier, time slot, road quality of the provider's base area and provider completion rate) and return the providers in descending order of score. | Must | Model evaluation (5.3.4); API tests |
| FR-06 | The system shall return with each ranked provider a plain-language explanation of the main factors behind the score, together with the estimated travel time and the distance. | Must | Inspection; API tests |
| FR-07 | The system shall let a client narrow the ranked results by maximum hourly rate, minimum rating and verified providers only, without changing any score or the order of providers. | Should | API filter tests |
| FR-08 | The system shall list providers that have no job history separately so that new providers are not hidden by the ranking. | Should | Cold-start tests |

| ID | Requirement | Priority | Verification |
|---|---|---|---|
| FR-09 | The system shall create a booking when a client selects a provider, shall recompute the score on the server instead of trusting a value sent by the browser, and shall notify both parties. | Must | Booking tests |
| FR-10 | The system shall let a provider accept, decline or complete a booking and let either party cancel one. It shall require a reason when a confirmed booking is cancelled and shall reject any status change the booking's current status does not allow. | Must | Booking tests (status transitions) |
| FR-11 | The system shall let either party propose a new date or time slot for a pending or confirmed booking and let the other party accept or decline it. A change shall be applied only on acceptance and only if the provider is still available. | Could | Rescheduling tests |
| FR-12 | The system shall let a provider set the weekdays and time slots in which the provider accepts work. | Must | Availability tests |
| FR-13 | The system shall let a client rate a completed booking from 1 to 5 with an optional comment, once only, and shall update the provider's displayed rating from these reviews. | Should | Review tests |
| FR-14 | The system shall create an in-app notification for each booking event and let the recipient mark it as read. | Should | Notification tests |
| FR-15 | The system shall show a user's profile in read-only form and let the user update an exact location. | Should | API tests |
| FR-16 | The system shall create administrator accounts only through a seeding script and never through public registration. | Must | Code inspection; authentication tests |
| FR-17 | The system shall let an administrator verify provider identity, monitor system performance and trigger model retraining. | Should | Administrator API tests (5.4.2) |

### 4.2.2 Non-Functional Requirements

Non-functional requirements describe how well the system must operate and the constraints under which it works. Only the quality areas that matter for this project are included, and no numerical threshold has been invented where none could be justified. Where a threshold depends on measurements in the deployed environment, the requirement says so and the value is fixed in Chapter 5. The requirements are listed in Table 4.2.

*Table 4.2 Non-functional requirements*

| ID | Requirement | Quality area | Verification |
|---|---|---|---|
| NFR-01 | On the same held-out test set, the proposed model shall achieve a lower RMSE and a lower MAE than both the distance-only baseline and the rating-only baseline. | Functional suitability (model quality) | Experiment (5.3.4) |
| NFR-02 | Passwords shall never be stored or logged in plain text. Only bcrypt hashes shall be stored. | Security (confidentiality) | Code inspection; schema; authentication tests |
| NFR-03 | Role-specific pages and operations shall be offered only to the matching role. Because sessions are not yet verified on the server, this is a user-experience guard and is not claimed as production security. | Security (authorisation) | End-to-end tests; documented limitation |
| NFR-04 | The database shall enforce its own integrity rules: consistency between a user's role and the linked record, permitted booking statuses, ratings from 1 to 5, reliability scores from 0 to 1, and at most one open reschedule proposal per booking. | Reliability (data integrity) | Constraint-violation checks (5.3.1) |
| NFR-05 | A recommendation request shall return within a response-time limit that is fixed after benchmarking the deployed environment. No limit is assumed in advance. | Performance efficiency | Latency measurement (5.4) |
| NFR-06 | Every displayed reliability score shall be accompanied by its explanation so that the ranking can be interpreted by the client. | Transparency and explainability | Inspection; user evaluation |
| NFR-07 | The web application shall be usable on desktop and mobile browsers and installable to a phone's home screen. | Usability; compatibility | End-to-end tests; responsive check |
| NFR-08 | The environment shall be reproducible: dependency versions pinned exactly, model training seeded, and the database built from the schema file and migration scripts. | Maintainability; portability | Clean rebuild (5.2.3) |
| NFR-09 | Backend behaviour shall be covered by automated tests that run from a clean checkout. | Maintainability (testability) | Test run (5.4) |
| NFR-10 | Where no data source exists for an item, the interface shall show an explicit empty or "coming soon" state and shall not display invented figures. | Functional correctness | Inspection of dashboards |

### 4.2.3 Data Requirements

Data requirements describe the data the system needs, produces and protects. They are particularly important in this project because the central output, the arrival reliability score, is learned from data. Table 4.3 lists them.

*Table 4.3 Data requirements*

| ID | Name | Requirement |
|---|---|---|
| DR-01 | Training data | The model shall be trained on 3,000 synthetic historical bookings whose target, arrival_reliability_score, is a continuous value between 0.0 and 1.0. Because the target is continuous, the task is regression and is evaluated with RMSE, MAE and $R^2$. |
| DR-02 | Leakage control | Columns that are known only after a job has finished (actual_travel_min, delay_min, on_time, job_completed and client_rating_given) shall never be used as model inputs. |
| DR-03 | Reference data | The system shall hold 30 named Nairobi areas with a centroid stored as a PostGIS point (SRID 4326) and a road-quality class, congestion multipliers and average speeds per corridor, time slot and day type, and provider profiles with availability. |
| DR-04 | Input validation | The time slot shall be one of morning_rush, midday, evening_rush or weekend_day, the day type shall be weekday or weekend, and the job date shall be a Nairobi calendar date from today to 60 days ahead. Unknown values shall be rejected rather than silently replaced by a default. |
| DR-05 | Separation of live and training data | Bookings made in the application shall be stored in a bookings table. The historical_bookings table is training data only and the application shall never write to it. |
| DR-06 | Missing history | A provider without a completion rate shall be scored using the platform median as a prior and shall be flagged as new. Reviews shall be created only by real completed bookings and shall never be back-filled with synthetic values. |
| DR-07 | Privacy and secrets | The dataset shall contain no real persons. Passwords shall be hashed, and database credentials and other secrets shall be kept outside the repository. |
| DR-08 | Retention | No deletion or retention policy is implemented for bookings, reviews or notifications. This is recorded as a limitation to be addressed in future work. |

### 4.2.4 Interface and Integration Requirements

Interface and integration requirements describe how users, software components and external services interact with the system. Table 4.4 lists them.

*Table 4.4 Interface and integration requirements*

| ID | Name | Requirement |
|---|---|---|
| IR-01 | Client and API | The Next.js web application shall communicate with the FastAPI service through JSON over HTTPS. The API shall accept browser requests only from the origins listed in its configuration. |
| IR-02 | API and database | The API shall read reference data from PostgreSQL with PostGIS. When no database is reachable it may fall back to the source CSV files, which allows prediction previews to run but not booking, profile or review functions. |
| IR-03 | Model interface | The trained model shall be a single scikit-learn pipeline, comprising the encoding step and the XGBoost regressor, that is loaded once when the service starts. It shall receive the five features of one client, provider and time-slot combination and return one score, which the booking service limits to the range 0 to 1. |
| IR-04 | Location and maps | A client or provider shall be able to give an exact location by placing a pin on an OpenStreetMap-based map. The service shall resolve the coordinates to the nearest named area and shall treat a point more than 8 km from every area centroid as outside the area covered by the traffic and road data. |
| IR-05 | Error handling | The API shall answer with distinct status codes for an unknown record (404), a validation failure (422), a conflict with the current state of a booking or availability (409) and an ownership failure (403). The interface shall show loading and error states instead of an empty page when the API is slow or unreachable. |
| IR-06 | Responsive interface | The interface shall adapt to phone and desktop widths and shall provide a web app manifest so that it can be installed. |

## 4.3 System Analysis Diagrams

This section presents the diagrams used to analyse the actors of the system, its boundary and the order of the interactions in its central task. Each diagram is introduced by a short narrative, shown with a numbered caption and interpreted afterwards.

### 4.3.1 Use Case Diagram

A use case diagram gives a high-level view of how external actors interact with the system and which services the system exposes. Figure 4.3.1 shows the system boundary, three external actors (the Client, the Service Provider and the System Administrator) and the use cases available to each. It addresses requirements FR-01 to FR-17 and shows the complete intended scope of the project, including administrator functions that are deferred in this release.

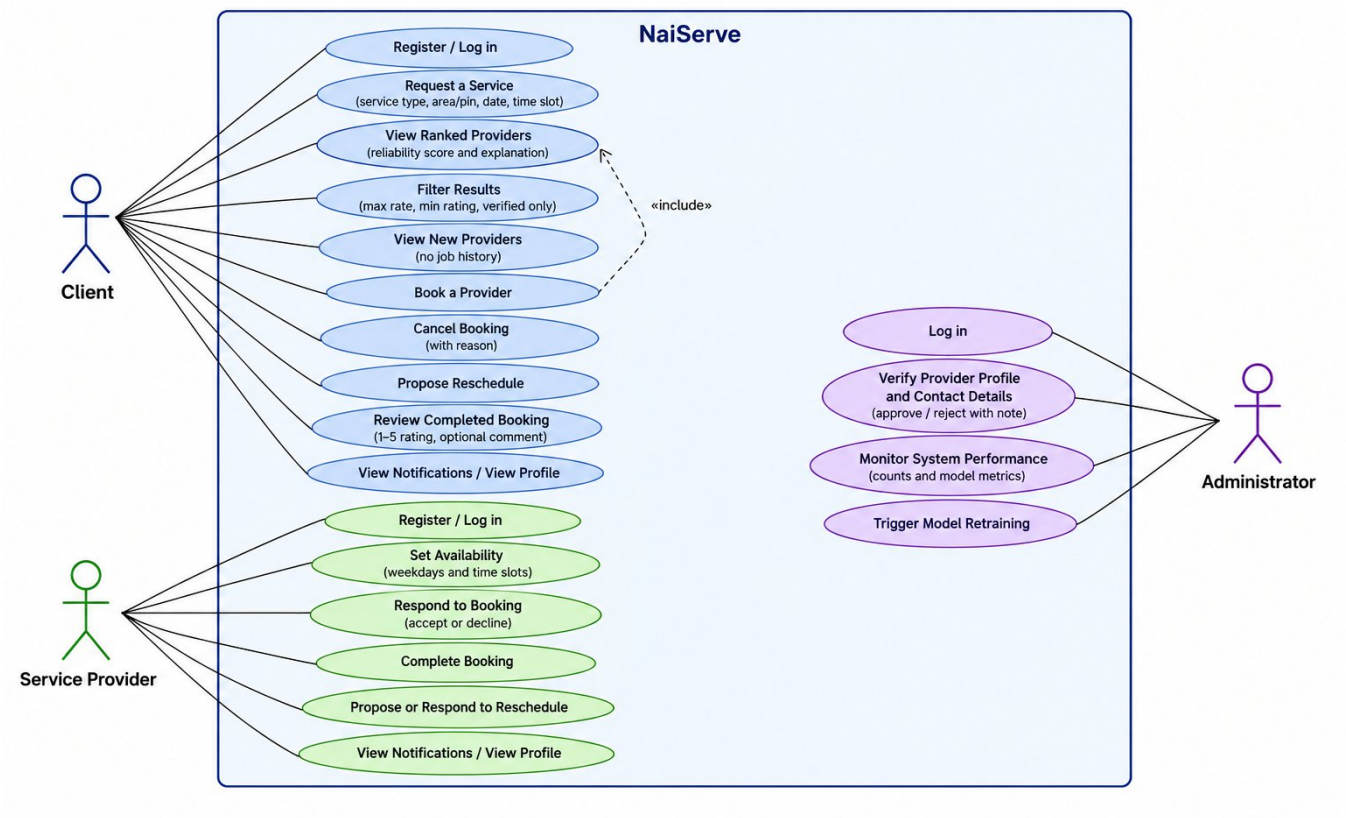



*Figure 4.3.1 Use Case Diagram*

The Client actor is associated with the use cases that make up the request-to-booking journey: Submit Booking Request, View Ranked Provider Recommendations, View Provider Explanation, Confirm Booking and Rate Completed Service. Compute Reliability Score is shown inside the boundary because it is the behaviour that distinguishes this system from a distance-based platform. In the implementation it is performed by the system and is not initiated by the Client, so it should be read as a system behaviour triggered by the request. The Service Provider actor registers a profile, sets an availability schedule, receives booking notifications, accepts or declines bookings and updates profile information. The System Administrator is outside the boundary of the client and provider journeys and is associated with Verify Provider Identity, Monitor System Performance and Trigger Model Retraining.

The «include» arrows converge on Confirm Booking and on Accept or Decline Booking. They are used here to show that the steps on the left lead to a single goal, namely a confirmed booking on the client side and a decision on the provider side. Strictly, UML reserves «include» for behaviour that a base use case always invokes, so the diagram is best read as a goal-oriented grouping of steps rather than a formal decomposition. The note in the diagram states that the

system gives no guarantee of service quality or safety, because the scores are predictions learned from historical data.

### 4.3.2 Sequence Diagram

A sequence diagram shows how objects exchange messages over time. Figure 4.3.2 follows the most important scenario in the system: a client submits a booking request and receives a ranked list of providers. It involves the browser, the Next.js frontend, the FastAPI service, the feature engineering module, the XGBoost model and the PostgreSQL database, and it addresses requirements FR-03 to FR-06 and IR-02 and IR-03.

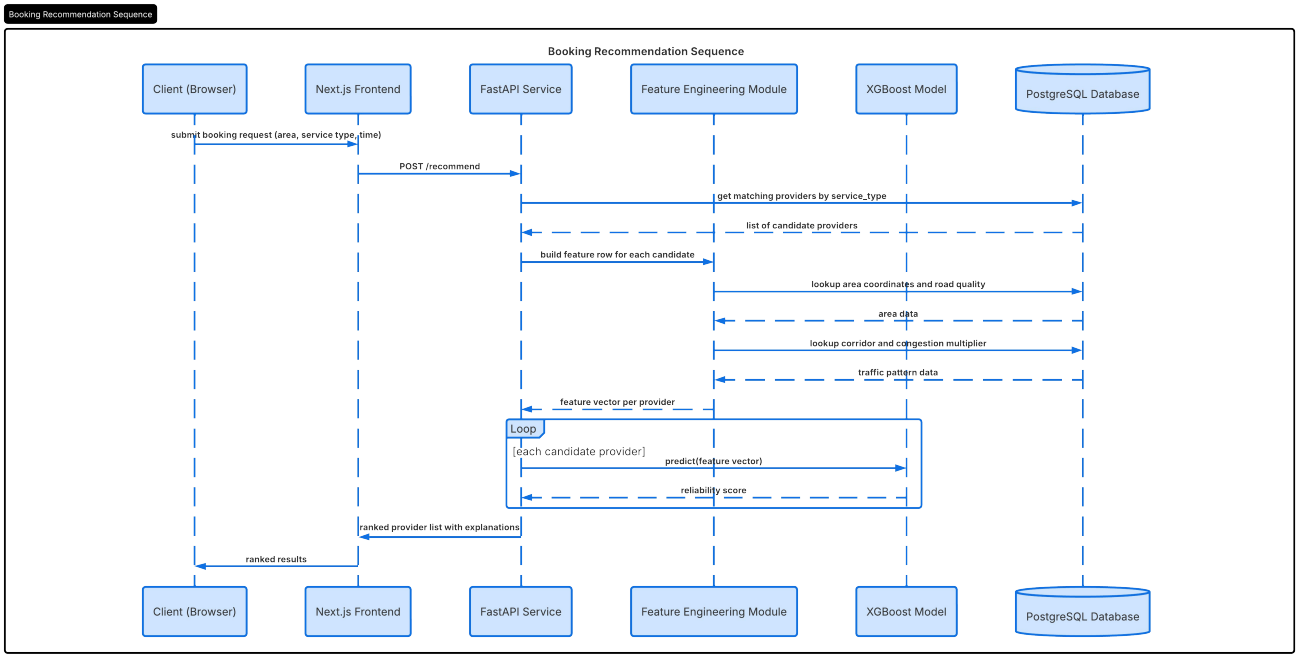



*Figure 4.3.2 Sequence Diagram: Booking Recommendation*

The client submits an area, a service type and a time, and the frontend forwards them to the POST /recommend endpoint. The service retrieves the candidate providers for the requested service type and asks the feature engineering module to build one feature row for each candidate. To do this the module looks up the coordinates and road quality of the areas involved and the congestion multiplier of the relevant corridor, and combines them with the provider's completion rate. The loop in the lower part of the diagram shows that the model is invoked once per candidate and returns one reliability score each. The service then sorts the candidates by score, attaches an explanation to each and returns the ranked list through the frontend to the client.

Two details of the implementation are not drawn in the diagram. First, providers without a

matching availability record are removed before the loop (FR-04), so the model scores only providers who can actually take the job. Second, the reference tables are loaded into the memory of the service when it starts, so the lookups drawn as database calls are served from memory, while bookings, ratings and profiles are read from PostgreSQL when a request arrives.

## 4.4 System Design Diagrams

The design diagrams describe how the system is organised to satisfy the requirements analysed above. They cover the technical structure of the solution, its classes and data, and the user interface. The same narrative, diagram and interpretation sequence is used.

### 4.4.1 System Architecture

The architecture diagram in Figure 4.4.1 gives a high-level view of the technology stack and the direction of data flow. It organises the system into a data layer, a processing and logic layer, a presentation and consumption layer, an integration layer and a set of cross-cutting support services. It addresses requirements FR-05, NFR-02, NFR-07 and IR-01 to IR-04.

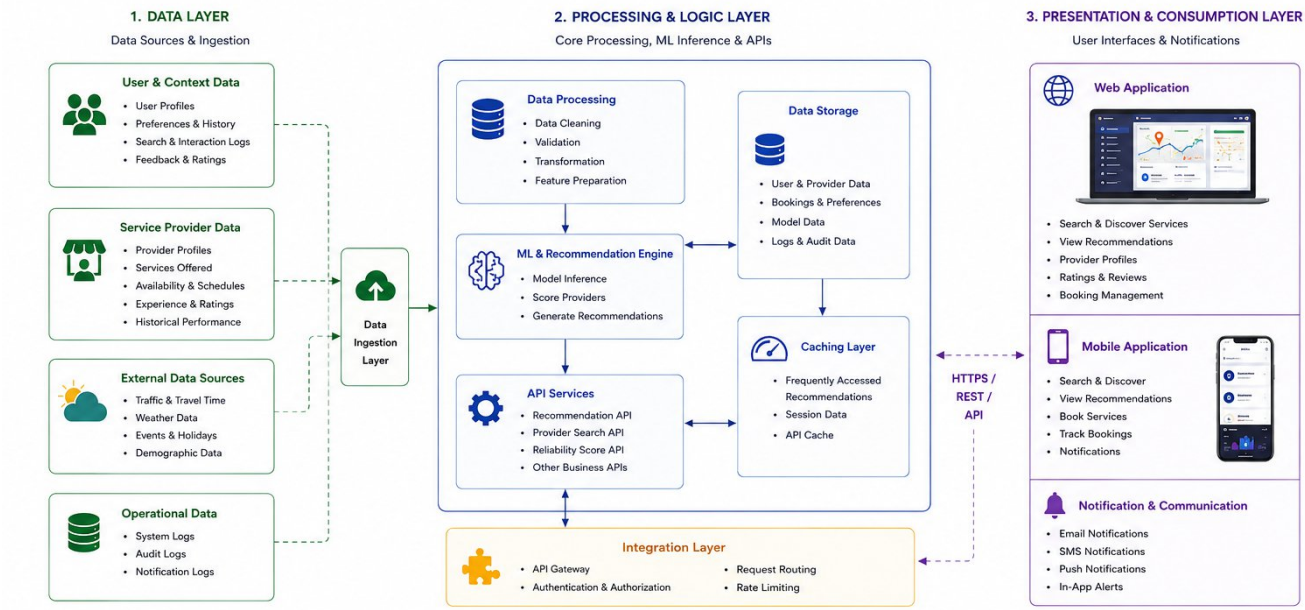



*Figure 4.4.1 System Architecture Diagram*

In the data layer, user, provider, traffic and area data are ingested and held in PostgreSQL with PostGIS. The processing layer contains the data processing and feature preparation steps, the ML and recommendation engine that applies the trained XGBoost pipeline, and the API services that expose recommendations and reliability scores. The presentation layer is the responsive Next.js

web application, which gives clients and providers their search, ranking, booking and notification screens. The integration layer carries requests between the tiers over HTTPS and REST, and authentication and role-based access control run across all of them. Separating the model behind an API means that the model can be retrained or replaced without changing the frontend.

The diagram shows the target architecture, and the prototype realises a subset of it. Elements that the project scope excludes or that are not yet built are the native mobile application (the web application is responsive and installable instead), e-mail, SMS and push notifications (only in-app notifications exist), external weather, events and live traffic feeds, and the caching layer.

### 4.4.2 Class Diagram

The class diagram in Figure 4.4.2 describes the static structure of the recommendation pipeline: the main classes, their attributes and operations, and the relationships between them. It supports requirements FR-03, FR-05 and FR-06 and shows how a booking request becomes a feature vector and then a ranked provider.

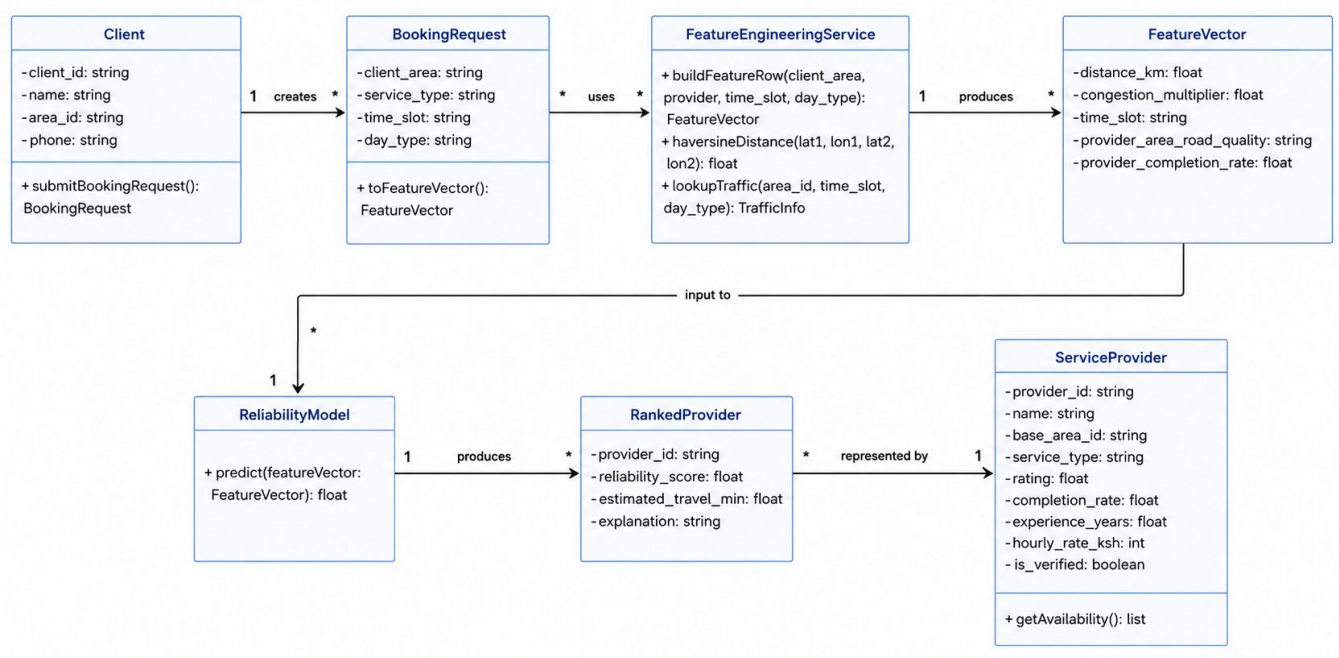



*Figure 4.4.2 Class Diagram*

A Client creates many BookingRequests, each holding the client area, the service type, the time slot and the day type. The FeatureEngineeringService uses a request together with a provider to produce a FeatureVector, whose five attributes (distance_km, congestion_multiplier, time_slot, provider_area_road_quality and provider_completion_rate) are exactly the inputs of the model.

The operations buildFeatureRow, haversineDistance and lookupTraffic correspond to the functions build_feature_row, haversine_km and lookup_traffic of the feature engineering module. The ReliabilityModel receives a FeatureVector and produces a RankedProvider carrying the reliability score, the estimated travel time and the explanation, and each RankedProvider is represented by one ServiceProvider.

The diagram is conceptual. The backend is written as Python functions and Pydantic schemas rather than as a hierarchy of classes, so each class here is realised as a module function or a request and response schema with the same responsibility. Bookings, reviews and notifications are not drawn because they belong to the transactional part of the system, which is modelled in the entity relationship diagram and the schema.

### 4.4.3 Entity Relationship Diagram

The entity relationship diagram (ERD) in Figure 4.4.3 shows the entities of the data model, their attributes and their relationships at a conceptual level. It supports requirements DR-01, DR-03 and DR-05.

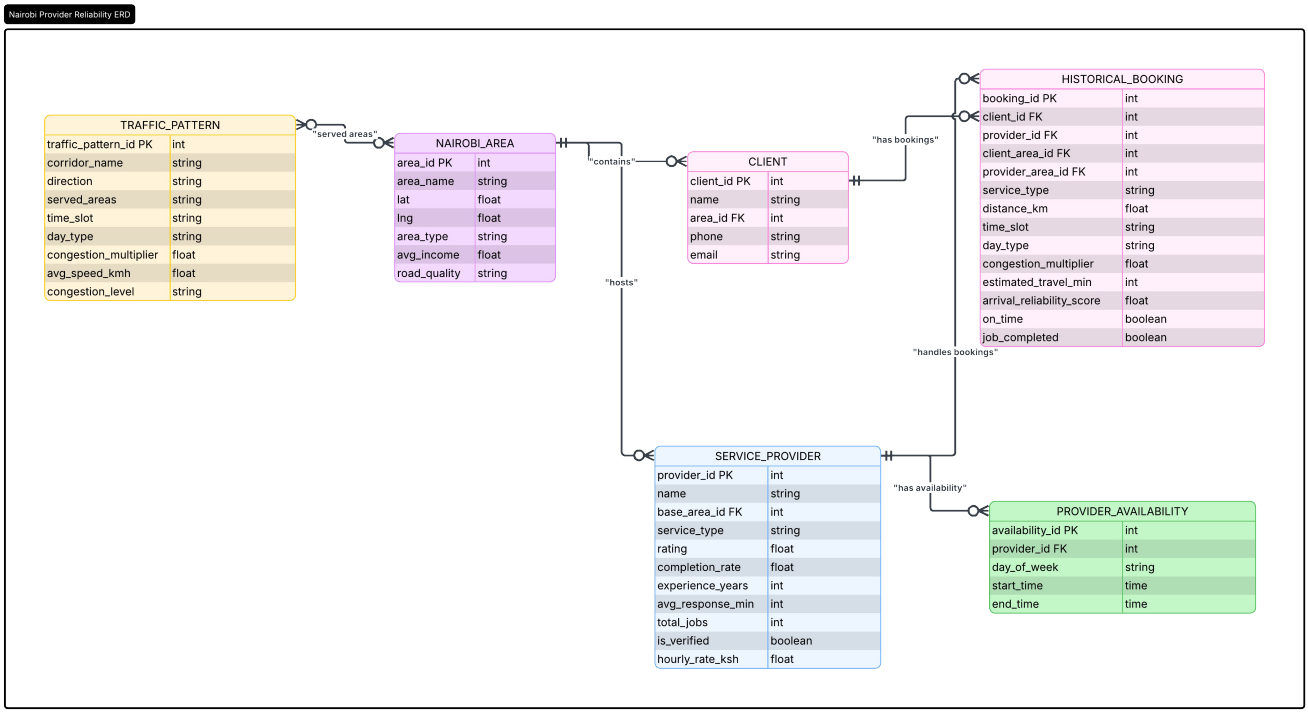



*Figure 4.4.3 Entity Relationship Diagram*

NAIROBI_AREA is the central reference entity. One area contains many clients and hosts many service providers, so the area supplies the road quality and the centroid used to compute distance.

A client has many historical bookings and a service provider handles many historical bookings and has many availability records. A TRAFFIC_PATTERN is related to many areas through its served areas, and an area is served by many traffic patterns, which is a many-to-many relationship. The HISTORICAL_BOOKING entity joins a client, a provider and two areas to the features and the arrival reliability score and is therefore the source of the training data.

### 4.4.4 Logical Database Schema

The logical database schema in Figure 4.4.4 translates the ERD into tables, attributes, primary and foreign keys and cardinalities. It also resolves the many-to-many relationship that the ERD records as a list of served areas. The figure shows the seven data tables that were designed first. Table 4.5 lists all thirteen application tables of the current schema, which adds the tables that support accounts, live bookings, notifications, reviews and rescheduling.

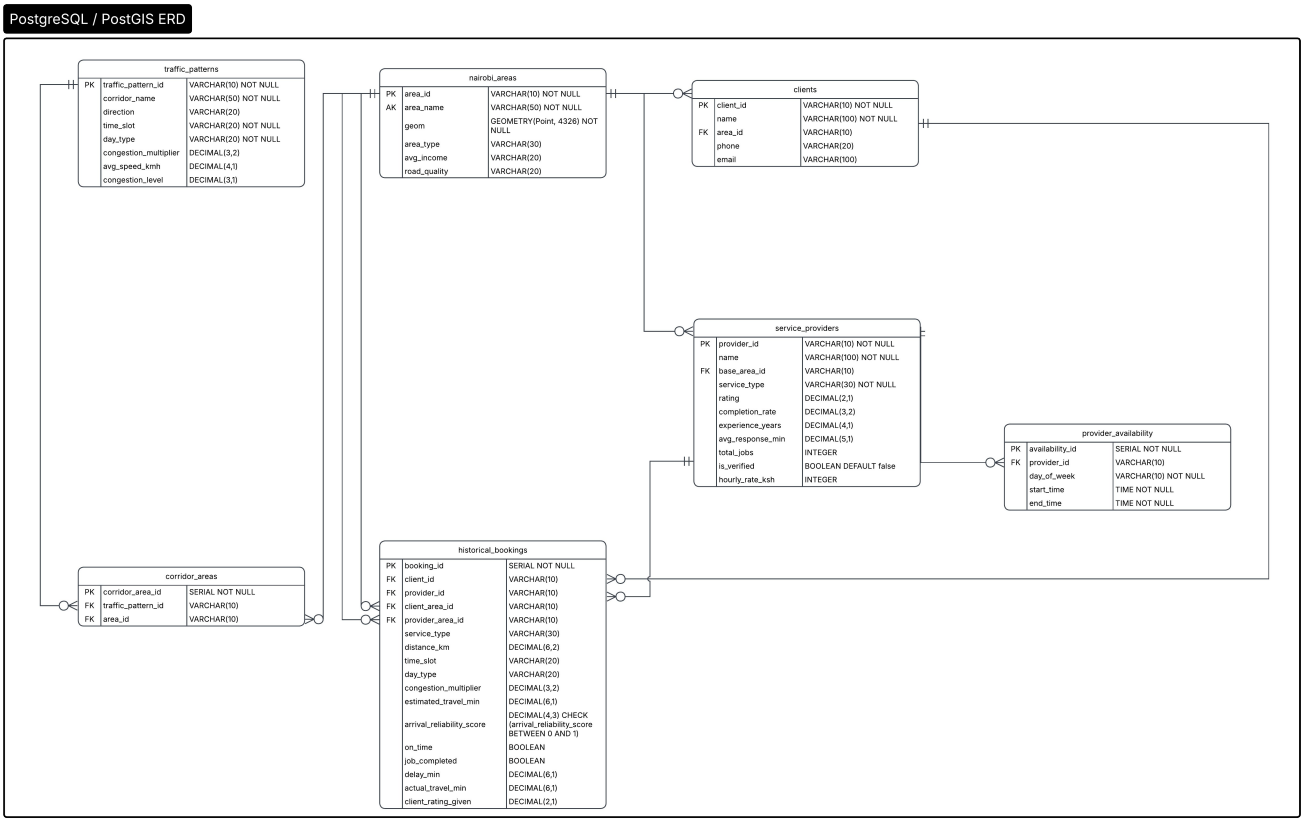



*Figure 4.4.4 Logical Database Schema*

The most important design decision is the corridor_areas table. The source data stores the areas served by a corridor as one delimited string, which breaks first normal form and makes a query such as “which corridors serve this area” impossible to index. The junction table holds one row

per traffic pattern and area pair, with a uniqueness constraint on the pair, and both columns are foreign keys. Provider and client positions are stored as PostGIS point geometries (SRID 4326) with GIST indexes so that spatial queries are efficient, while distances for the model are computed with the Haversine formula. Integrity is enforced in the database itself: a check constraint requires that a user has exactly the foreign key that matches the role, and the reliability score in historical_bookings is constrained to the range 0 to 1.

*Table 4.5 Application tables of the logical schema*

| **Table** | **Purpose** | **Key relationships** |
|---|---|---|
| nairobi_areas | Thirty named areas with a centroid point, area type, income band and road quality. | Referenced by clients, service_providers, corridor_areas and historical_bookings. |
| clients | Registered clients with a home area and an optional exact location. | area_id references nairobi_areas. |
| service_providers | Provider profile: service type, rating, completion rate, experience, response time, total jobs, verification flag, hourly rate and exact base location. | base_area_id references nairobi_areas. |
| traffic_patterns | Congestion multiplier, average speed and congestion level per corridor, time slot and day type. | Linked to areas through corridor_areas. |
| corridor_areas | Junction table resolving the many-to-many relationship between traffic patterns and areas. | traffic_pattern_id and area_id are foreign keys; the pair is unique. |
| provider_availability | Weekday and working hours of each provider. | provider_id references service_providers. |
| historical_bookings | Synthetic past bookings with the features and the reliability score. Training data only. | References a client, a provider and two areas. |
| users | Login accounts with a bcrypt hash and a role. A check constraint ties the role to exactly one linked record. | client_id or provider_id, according to the role. |
| bookings | Live requests made in the application, with status, job date, stored score, location and cancellation details. | References a client and a provider. |
| notifications | Event-triggered messages with a read flag. | user_id references users. |
| booking_reviews | One rating from 1 to 5 and an optional comment for each completed booking. | booking_id is both primary key and foreign key. |
| booking_reschedules | Proposals to move a booking, with the previous date and slot. At most one proposal per booking can be pending. | booking_id references bookings. |

| Table | Purpose | Key relationships |
|---|---|---|
| provider_identity_reviews | Administrator decision on a provider's profile and contact details, with reviewer, note and time. | provider_id references service_providers; reviewed_by references users. |

### 4.4.5 Wireframes

Wireframes are low-fidelity blueprints of the user interface. They fix the layout, the information shown and the interactive elements of each screen without deciding colour or typography. The wireframes below cover the client journey from the landing page to the booking confirmation, the provider onboarding and availability screens, the provider dashboard and the mobile layout of the main client screens. They address requirements FR-03, FR-06, FR-09, FR-10, FR-12 and NFR-06. The text, names and figures shown in the wireframes are placeholders used to show layout and are not data.

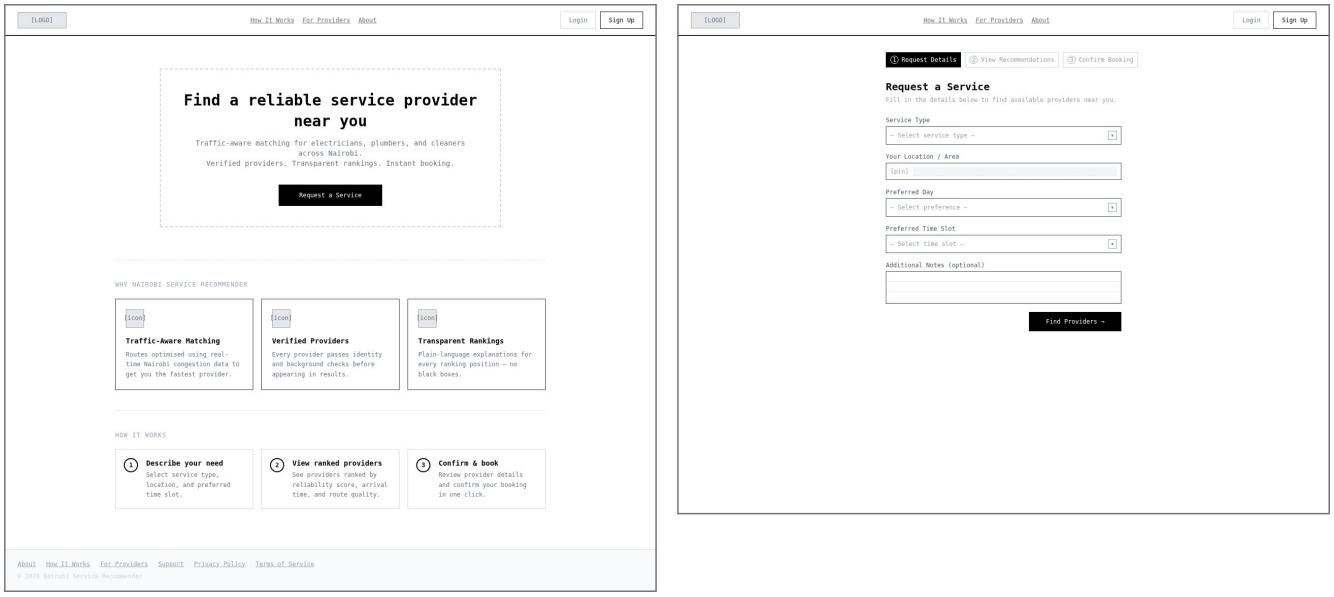



*Figure 4.4.5 Wireframes: landing page (left) and service request form (right)*

The landing page states the value proposition, offers a single call to action and explains in three steps how the service works. The request form follows with only the inputs the model needs: the service type, the location, the preferred day and the time slot, plus an optional note. Keeping the form short supports the objective of producing a ranking from a few contextual inputs.

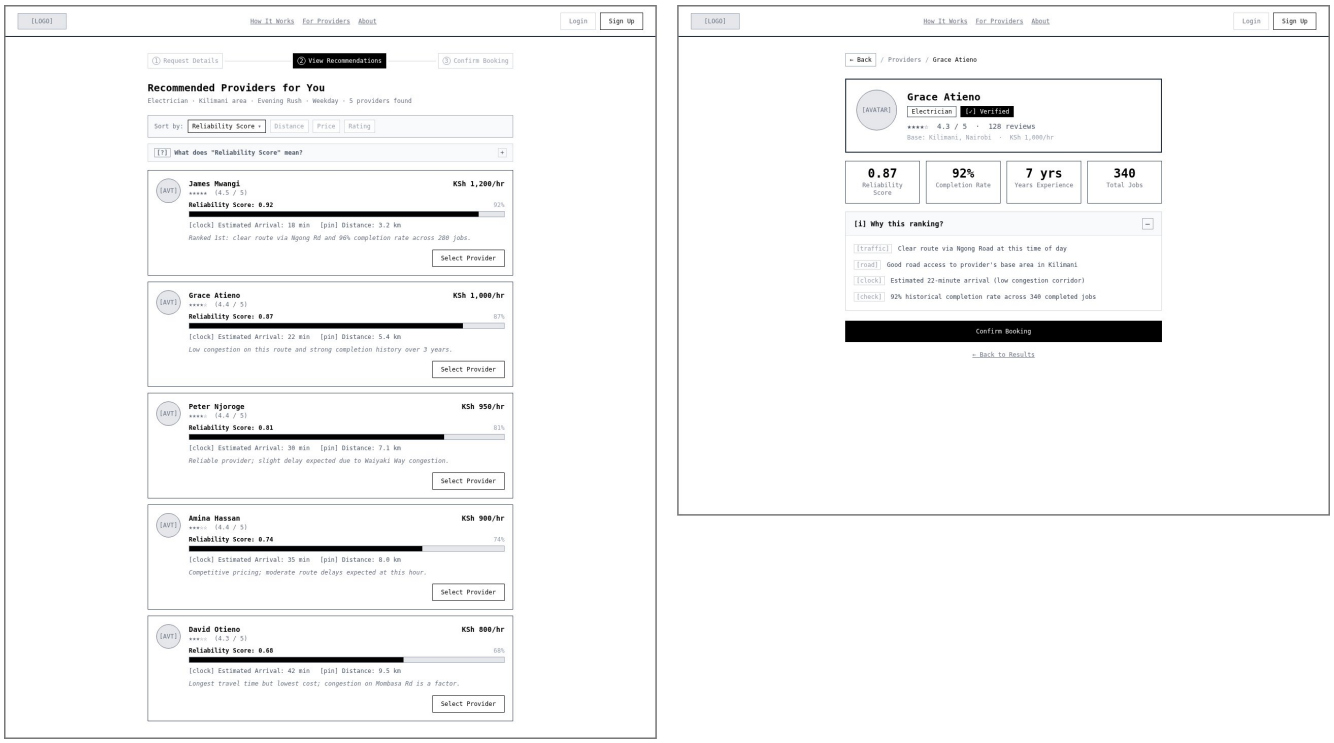



*Figure 4.4.6 Wireframes: ranked results (left) and provider detail with explanation (right)*

The results screen is the centre of the interface. Each provider appears as a card with a reliability score and a bar, the estimated arrival time, the distance, the hourly rate and a one-line reason, so that a client can see why a provider with a longer distance may rank higher. An expandable panel explains what the reliability score means. The detail screen adds the provider's completion rate, years of experience and total jobs and a "Why this ranking?" panel that lists the factors, which is the explainability layer described in Section 3.3.4.

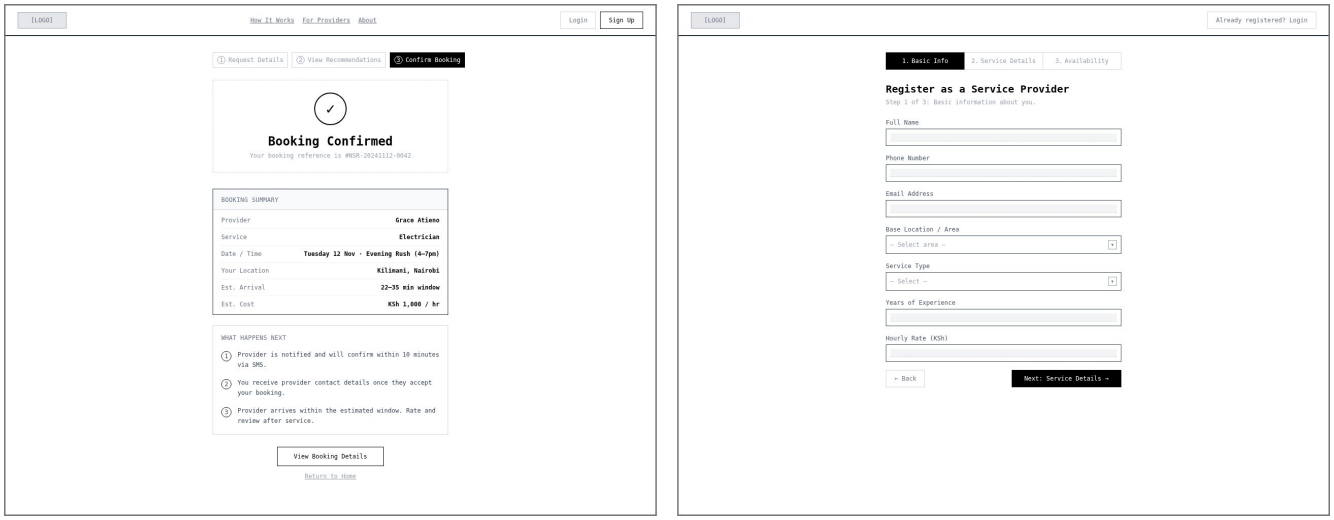



*Figure 4.4.7 Wireframes: booking confirmation (left) and provider registration (right)*

The confirmation screen summarises the booking and states what happens next, namely that the provider is notified, confirms, and arrives within the estimated window. Provider registration is split into steps so that a new provider is asked for basic information first, then service details and finally availability.

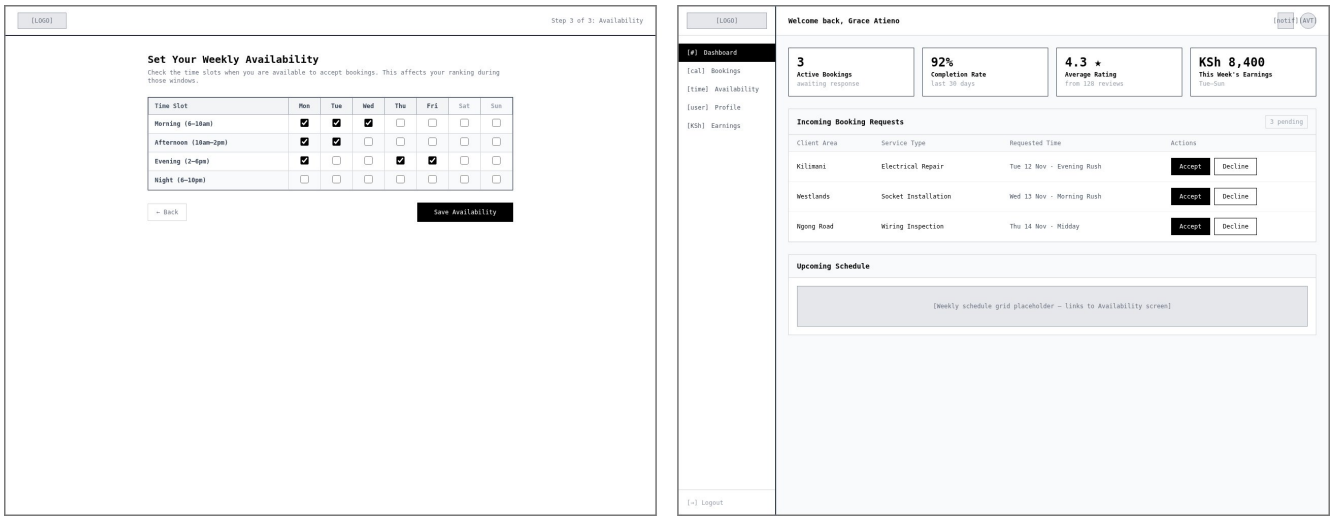



*Figure 4.4.8 Wireframes: provider availability (left) and provider dashboard (right)*

The availability screen is a grid of time slots against weekdays, which gives the provider direct control over the records that the availability filter (FR-04) depends on. The provider dashboard combines summary figures, incoming booking requests with accept and decline actions, and an upcoming schedule. Figures such as earnings appear in the wireframe only to show the layout; in the implemented system an item with no backing data is shown as an empty or “coming soon” state (NFR-10).

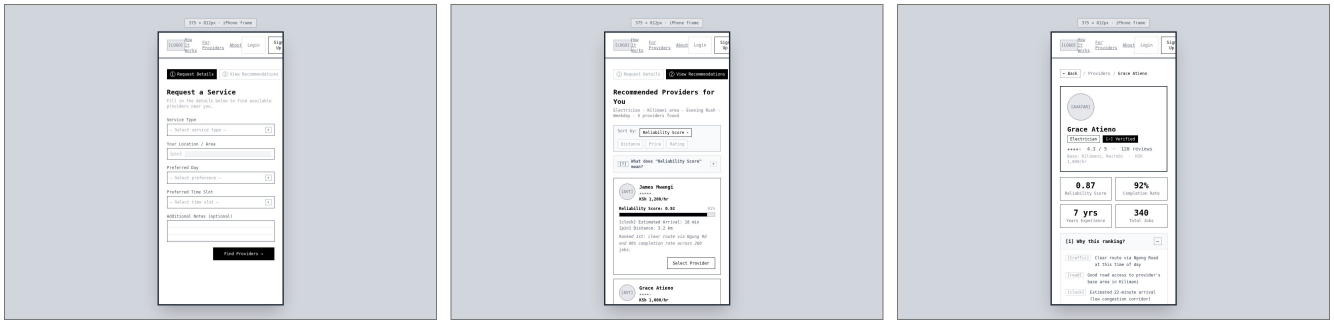



*Figure 4.4.9 Wireframes: mobile layout of the request, results and provider detail screens*

The mobile wireframes show that the same three screens stack into a single column and keep the primary action within reach. They respond to the scope decision that the system is delivered as a responsive web application accessible on desktop and mobile browsers, rather than as a native

mobile application. The screens as built are shown in Figures A2.1 to A2.3 in Appendix A2.

## 4.5 Requirements Traceability

Table 4.6 links the project objectives to the requirements, the diagrams that express the design and the verification evidence reported in Chapter 5. It allows a reader to check that every objective is supported by a requirement, a design element and a planned test.

*Table 4.6 Traceability from objectives to requirements, design and verification*

| **Objective or need** | **Requirements** | **Design evidence** | **Verification** |
|---|---|---|---|
| Objective i: identify the factors that influence reliability | DR-01, DR-03, FR-05 | Class diagram (FeatureVector); ERD and schema (traffic_patterns, nairobi_areas) | Feature importance analysis (5.3.4) |
| Objective ii: review existing platforms and identify gaps | FR-05, FR-06 | Use case diagram; architecture diagram | Gap analysis in Chapter 2; comparison with distance and rating baselines |
| Objective iii: develop a model that predicts arrival reliability | FR-03 to FR-06, DR-01, DR-02, IR-03 | Sequence diagram; class diagram; architecture diagram | Model training and evaluation (5.3) |
| Objective iv: evaluate against distance-based and rating-based approaches | NFR-01, DR-02 | Sequence diagram; schema (historical_bookings) | Test-set RMSE, MAE and $R^2$ (5.3.4) |
| General objective: a working recommendation system for clients and providers | FR-01 to FR-16, NFR-02 to NFR-10, IR-01 to IR-06 | Use case, architecture, ERD, schema and wireframes | Automated and end-to-end tests; user evaluation (5.4) |

This chapter specified what the system must do and how it is structured. Chapter 5 reports how it was implemented, the environment in which it was built, and the evidence collected so far.

# Chapter Five: System Implementation, Testing and Evaluation

## 5.1 Introduction

This chapter explains how the design approved in Chapter 4 was implemented, the environment in which it was built and deployed, and the evidence that the implementation works. Because the project is an AI-based information system, the chapter follows the route that links the database, the dataset, the model and the application. Section 5.2 describes the implementation environment. Section 5.3 describes the system implementation, from the database and the dataset to model training and the recommendation service, and records where the implementation departed from the design. Section 5.4 covers testing and evaluation, Section 5.5 the version-control evidence, Section 5.6 the use of AI-assisted tools, and Section 5.7 the evaluation results, discussion and limitations.

Unless stated otherwise, every figure in Sections 5.2 and 5.3 was reproduced from the project repository using a clean PostgreSQL and PostGIS container built from the schema file and the migration script. The training data are synthetic, so the results describe the behaviour of the system on simulated Nairobi data and must not be read as measurements of real providers. The evidence in Section 5.4 was collected from the committed repository together with the administrator functions, which were in the working tree and not yet committed, as explained in Section 5.5.

## 5.2 Implementation Environment

The implementation environment describes where the solution was developed, tested and deployed, and what is needed to reproduce it. The development environment is distinguished from the hosted environment, which is a free-tier demonstration deployment and not a production service.

### 5.2.1 Hardware and Deployment Environment

The system was developed, trained and tested on a single laptop. The model is small enough (3,000 records and five features) to be trained on the CPU without an accelerator. For demonstration, the three tiers of the architecture were deployed to hosted services, each in the tier that suited its workload: the web application on Vercel, the FastAPI and XGBoost service in a Docker container on Render, and the database on Neon. The Render and Neon instances were placed in Frankfurt to keep the two close to each other. Table 5.1 summarises the environment.

*Table 5.1 Implementation environment*

| Layer | Specification tested | Purpose | Evidence or source |
|---|---|---|---|
| Development workstation | AMD Ryzen 5 5625U, 12 logical CPUs, 14 GiB memory, Ubuntu 24.10, Docker 29.1.3 | Development, model training and automated tests | lscpu, free and /etc/os-release on the workstation |
| Local database | postgis/postgis:16-3.4 container (PostgreSQL 16, PostGIS 3.4) | Spatial and relational storage during development | SELECT PostGIS_Version() returned 3.4 |
| Hosted API | Render web service, Docker runtime, Frankfurt region, free plan, 512 MB memory; sleeps after 15 minutes idle, so the first request after a pause takes about 30 to 60 seconds | Runs the FastAPI service and XGBoost inference | render.yaml; docs/DEPLOYMENT.md |
| Hosted database | Neon PostgreSQL 16 with PostGIS, AWS eu-central-1 (Frankfurt) | Hosted data store for the deployed API | docs/DEPLOYMENT.md |
| Hosted frontend | Vercel project with root directory frontend, API address set through NEXT_PUBLIC_API_URL | Serves the Next.js web application | docs/DEPLOYMENT.md |
| Client device | Current desktop and mobile browsers. Automated tests drive desktop Chrome through Playwright. A minimum phone specification was not measured. | Runs the web application | tests/e2e/playwright.config.ts |

### 5.2.2 Software, Services and Configuration

Table 5.2 lists the software that was used to build and run the system, with the versions that were tested. The Python dependencies are pinned to exact versions in two files: requirements.txt for development and requirements-api.txt for the deployed image, which omits the demonstration, plotting and testing packages. The frontend versions are those declared in package.json, with the installed version of Next.js shown where it differs. Passwords, database credentials and other secrets are held in a local file that is excluded from version control and in the Render and Vercel dashboards, and none appears in the repository.

*Table 5.2 Software, libraries and services*

| Layer | Technology and version | Role in the system |
|---|---|---|
| Language and runtime | Python 3.12.7; Node.js 22.12.0 | Backend and machine learning code; frontend build and tooling |

| Layer | Technology and version | Role in the system |
|---|---|---|
| API | FastAPI 0.115.0 with Uvicorn 0.30.6 | Serves the recommendation, booking, account and notification endpoints |
| Machine learning | XGBoost 2.1.1; scikit-learn 1.5.2; pandas 2.2.3; joblib 1.4.2 | Feature handling, the encoding step and regressor of the pipeline, and model persistence |
| Database | PostgreSQL 16 with PostGIS 3.4; psycopg2-binary 2.9.9 | Relational and spatial storage |
| Security | bcrypt 4.2.0 | Password hashing |
| Frontend | Next.js 14.2.35 (declared ^14.2.15); React ^18.3.1; TypeScript ^5.5.4; Tailwind CSS ^3.4.13 | Web application, routing and styling |
| Maps | Leaflet 1.9.4; react-leaflet 4.2.1; OpenStreetMap tiles | Location picker and map display |
| Testing | pytest 8.3.3 with httpx 0.27.2; Playwright 1.59.1 | Backend tests; end-to-end browser tests |
| Delivery | Docker; Git and GitHub; Vercel, Render and Neon | Containers, version control and hosting |

### 5.2.3 Deployment and Reproducibility

The system can be rebuilt from the repository with the following steps. Steps 1, 2 and 5 and the backend test suite were carried out against a new, empty PostGIS container to confirm that the schema, the migration and the tests work from a clean start. The model figures in Section 5.3 were reproduced with the same pipeline definition that Step 4 uses. Table 5.3 lists the steps.

*Table 5.3 Steps to rebuild the system from the repository*

| Step | Component | Action and evidence |
|---|---|---|
| 1 | Database | Start a postgis/postgis:16-3.4 container and apply database/schema.sql, which enables PostGIS and creates the thirteen application tables. |
| 2 | Data | Run database/migrate_csv_to_postgres.py with the DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD variables set. The script loads the source CSV files into the reference and training tables and can be repeated with its --reset option. |
| 3 | Administrator | Run database/seed_admin.py with the ADMIN_EMAIL and ADMIN_PASSWORD variables to create the only kind of account that cannot be registered through the interface. |
| 4 | Model | Install the pinned dependencies and run finalize_model.py in ml-service. It trains the pipeline with a fixed random seed, prints the test and cross-validation errors and saves the model file. The Docker image used for the hosted API runs the same script during its build. |

| Step | Component | Action and evidence |
|---|---|---|
| 5 | Api | Start the service with Uvicorn and call GET /health. The check returned {"status":"ok","providers_loaded":200,"areas_loaded":30,"availability_loaded":1117}, which confirms that the model and the reference data were loaded. |
| 6 | Frontend | Run npm install and npm run dev in the frontend folder with NEXT_PUBLIC_API_URL pointing to the API. |

The additive migrations 002 to 007 in database/migrations upgrade a database that was created from an earlier version of the schema; a fresh installation uses schema.sql, which already contains their changes. With the database variables set, 182 backend tests passed and 13 were skipped by default. With RUN_DB_TESTS=1 the 13 PostgreSQL-isolated tests also ran, and all 195 backend tests passed. The suite was run again after the administrator functions were added, and all 205 backend tests passed.

## 5.3 System Implementation

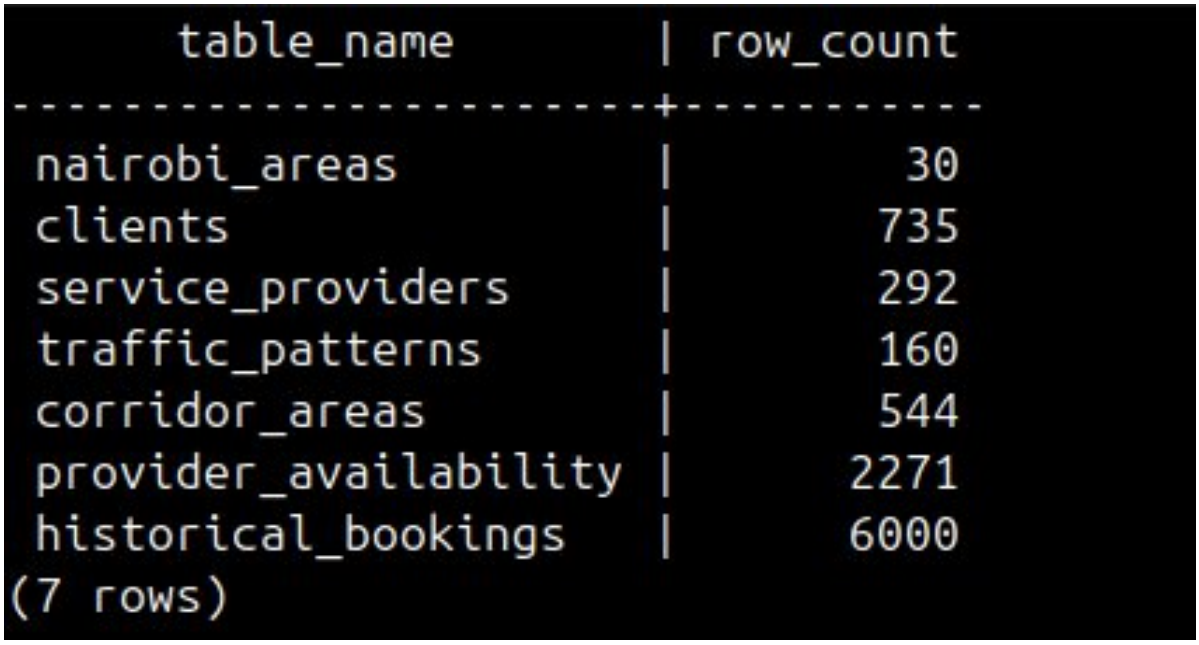
The implementation is described in the order of the architecture in Chapter 4: first the database, then the dataset and the model, and then the service that combines them. Each subsection identifies the requirements it implements and the evidence that it works.

### 5.3.1 Database and Data Implementation

The logical schema of Section 4.4.4 was implemented in PostgreSQL 16 with the PostGIS extension. The schema file creates thirteen application tables, GIST indexes on the point geometries of areas, clients and providers, and B-tree indexes on the foreign keys that the queries join on. Later requirements were met by additive migrations instead of changes to the original tables: exact locations (002), booking reviews (003), cancellation reasons (004), job dates (005) reschedule proposals (006) and provider identity reviews (007). Each migration can be applied again without harm.

The CSV files were loaded by the migration script. The pipe-delimited list of served areas in the traffic file was split into rows of the corridor_areas junction table, so that 160 traffic patterns produced 544 corridor and area pairs. Figure 5.3.1 shows the current PostgreSQL row counts captured from the running database: 30 areas, 735 clients, 292 service providers, 160 traffic patterns, 544 corridor-area mappings, 2,271 availability records and 6,000 historical bookings. These are the database contents at screenshot capture time, distinct from the original 3,000-record training dataset described in Section 5.3.2. The screenshot alone does not establish why

the counts have increased or that the model was retrained on those additional rows. The tables that hold application activity (users, bookings, notifications, booking_reviews and booking_reschedules) start empty and are filled only by real use of the application, in line with DR-05 and DR-06.



| table_name | row_count |
|---|---|
| nairobi_areas | 30 |
| clients | 735 |
| service_providers | 292 |
| traffic_patterns | 160 |
| corridor_areas | 544 |
| provider_availability | 2271 |
| historical_bookings | 6000 |

(7 rows)

*Figure 5.3.1 Current database row counts from PostgreSQL*

The integrity rules of requirement NFR-04 were checked by attempting to break them directly in the database. Inserting a user with the role client and a provider identifier failed with a violation of the constraint chk_users_role_matches_fk, and inserting a historical booking with a reliability score of 1.5 failed with a violation of the check constraint on arrival_reliability_score. In both cases the database rejected the row without any help from the application, which means that a defect in the API cannot store an inconsistent account or an impossible score.

### 5.3.2 Dataset Description

The training data are the 3,000 synthetic bookings in historical_bookings. They were generated by the rule-based simulation described in Section 3.2.1, with congestion, speeds and road quality calibrated to published information about Nairobi. Each record describes one booking: who booked whom, the service type, the day and time slot, the distance between the two locations, the congestion multiplier of the provider's corridor, the road quality of the areas, the provider's rating, completion rate and experience, the outcome of the job, and the arrival reliability score that is the target of the model. Table 5.4 summarises the dataset and Figure 5.3.2 shows three

records with the five model features and the target.

*Table 5.4 Summary of the training dataset*

| **Property** | **Value** |
|---|---|
| Source and licence | Synthetic, generated for this project from a rule-based simulation. It contains no real persons or bookings. |
| Size | 6,000 records with 36 columns, booked between 1 January 2024 and 23 June 2025 |
| Entities | 200 providers, 499 of the 500 clients, 30 areas and 160 traffic patterns |
| Service types | electrician 719; cleaner 661; plumber 639; technician 568; carpenter 413 |
| Time slots | midday 817; early_afternoon 605; evening_rush 593; mid_morning 508; morning_rush 477 |
| Day types | weekday 2,120; weekend 880 |
| Road quality of the provider's area | good 1,076; poor 1,020; moderate 904 |
| Distance (km) | mean 13.72; median 11.91; range 0.28 to 46.20 |
| Congestion multiplier | mean 2.37; range 1.04 to 4.00 |
| Target: arrival_reliability_score | mean 0.765; standard deviation 0.189; median 0.814; range 0.0 to 1.0. 9.7% of records are below 0.5 and 28.8% are at or above 0.9. |
| Data quality | No missing values in the five model features or the target; no duplicate booking identifiers |

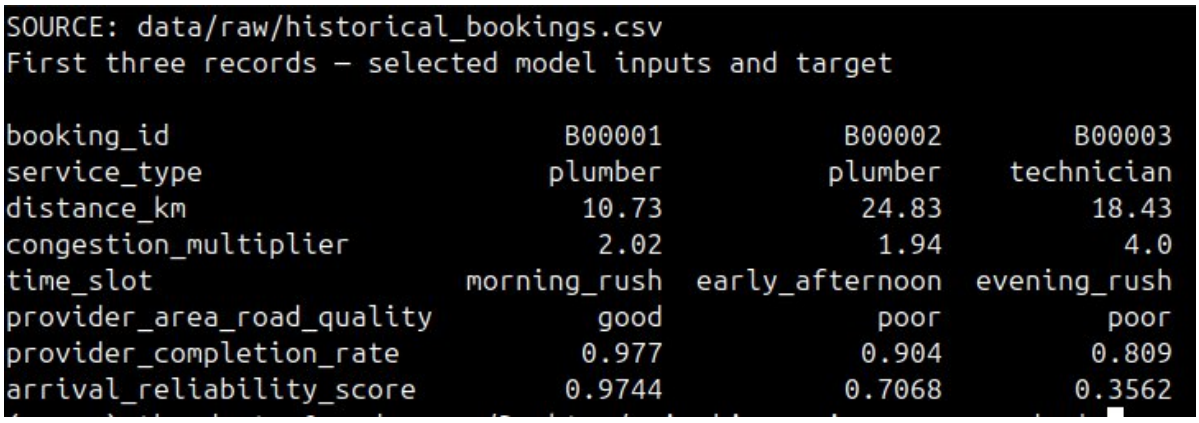



*Figure 5.3.2 Actual training dataset preview: model features and target*

The sample shows the pattern that the model has to learn. The third record combines a long distance, the highest congestion multiplier and a poor-quality area in the evening rush, and it has the lowest score, while the first combines a good area, moderate congestion and a high completion rate and has a score close to 1. The data cover five service types. The approved proposal named three; technician and carpenter were added in the simulation and are offered by

the system, and Section 1.5.1 has been updated to match.

### 5.3.3 Data Preparation and Partitioning

The model uses five input features, one for each conceptual pillar of the design: distance_km for spatial separation, congestion_multiplier for traffic, time_slot for time of day, provider_area_road_quality for road quality, and provider_completion_rate for provider history. The set was reduced from a larger candidate set after testing indicated that the five features matched the performance of the larger set. The estimated travel time is computed for display and explanation but is not a model input, because it is a function of distance and speed that the model already receives.

In line with DR-02, the columns that are known only after a job has ended (actual_travel_min, delay_min, on_time, job_completed and client_rating_given) were excluded, since using them would leak the outcome into the prediction. The data had no missing values in the model columns, so no imputation was needed for training. Preprocessing is part of the saved pipeline: the numeric features pass through unchanged and the two categorical features are one-hot encoded, with unseen categories ignored. Because the encoder is inside the pipeline, it is fitted on training data only, including inside each cross-validation fold.

The records were split at random into 2,400 for training (80%) and 600 for testing (20%) with a fixed seed of 42. The 600 test records were used only for the final comparison and not for tuning. Hyperparameters were selected by five-fold cross-validation on the training portion. The split is random across bookings and not grouped by provider, so the same providers appear in both parts. The test therefore measures how well new bookings of known providers are predicted, and it does not measure how well the model generalises to providers that it has never seen.

### 5.3.4 Model Training and Selection

Three conditions were compared, as set out in Section 3.2.3. Baseline A predicts the score from distance alone and Baseline B from the provider's rating alone, in both cases with a linear regression fitted on the training records. The proposed model is an XGBoost regressor inside the scikit-learn pipeline. A grid search with five-fold cross-validation over the learning rate (0.05 and 0.1), the maximum tree depth (3 and 5) and the minimum child weight (1 and 3) selected a learning rate of 0.05, a depth of 3 and a minimum child weight of 3. The other parameters kept their defaults and the random seed was 42. The untuned model, trained with default parameters, obtained an RMSE of 0.1583 and an MAE of 0.1247 on the test set in the training notebook, so

tuning reduced the error further.

Table 5.5 compares the conditions on the same 600 test records. The proposed model has an RMSE of 0.1417 and an MAE of 0.1160, which are 24.3% and 23.4% lower than those of Baseline A and 26.8% lower than those of Baseline B on both measures. Its $R^2$ is 0.4717, against 0.0776 for distance alone and 0.0141 for rating alone. The target is continuous, so RMSE, MAE and $R^2$ are the appropriate measures and accuracy is not used. These results satisfy requirement NFR-01 on this dataset.

*Table 5.5 Model comparison on the 600-record test set*

| **Condition** | **RMSE** | **MAE** | **$R^2$** |
|---|---|---|---|
| Baseline A: distance only (linear regression) | 0.1872 | 0.1514 | 0.0776 |
| Baseline B: provider rating only (linear regression) | 0.1936 | 0.1585 | 0.0141 |
| XGBoost with default parameters (training notebook) | 0.1583 | 0.1247 | Not reported |
| Proposed model: tuned XGBoost, five features | 0.1417 | 0.1160 | 0.4717 |

Five-fold cross-validation of the final pipeline over all 3,000 records gave a mean RMSE of 0.1414 with a standard deviation of 0.0020, and the fold errors ranged from 0.1387 to 0.1449. The mean is almost the same as the test RMSE, which indicates that the result does not depend on a favourable split. An earlier verification record shows a test RMSE of 0.1419 and an MAE of 0.1162, and the notebook reports a cross-validation mean of 0.1411. These differ from the values above only in the fourth decimal place and reflect small variation between runs and environments.

*Table 5.6 Relative importance of the model features (gain)*

| **Feature** | **Share of total importance** |
|---|---|
| congestion_multiplier | 51.5% |
| provider_completion_rate | 16.5% |
| time_slot (five encoded values) | 11.9% |
| provider_area_road_quality (three encoded values) | 10.3% |
| distance_km | 9.8% |

The importance values in Table 5.6 answer the first research question for this dataset: congestion on the provider's corridor carries about half of the model's explanatory weight, provider history comes second, and straight-line distance, the factor on which current platforms rank providers, contributes under a tenth. This agrees with the low $R^2$ of the distance baseline. The values should not be read as causal effects in Nairobi, because the data were produced by a simulator in which

these relationships were built in, and gain-based importance depends on how the features are correlated.

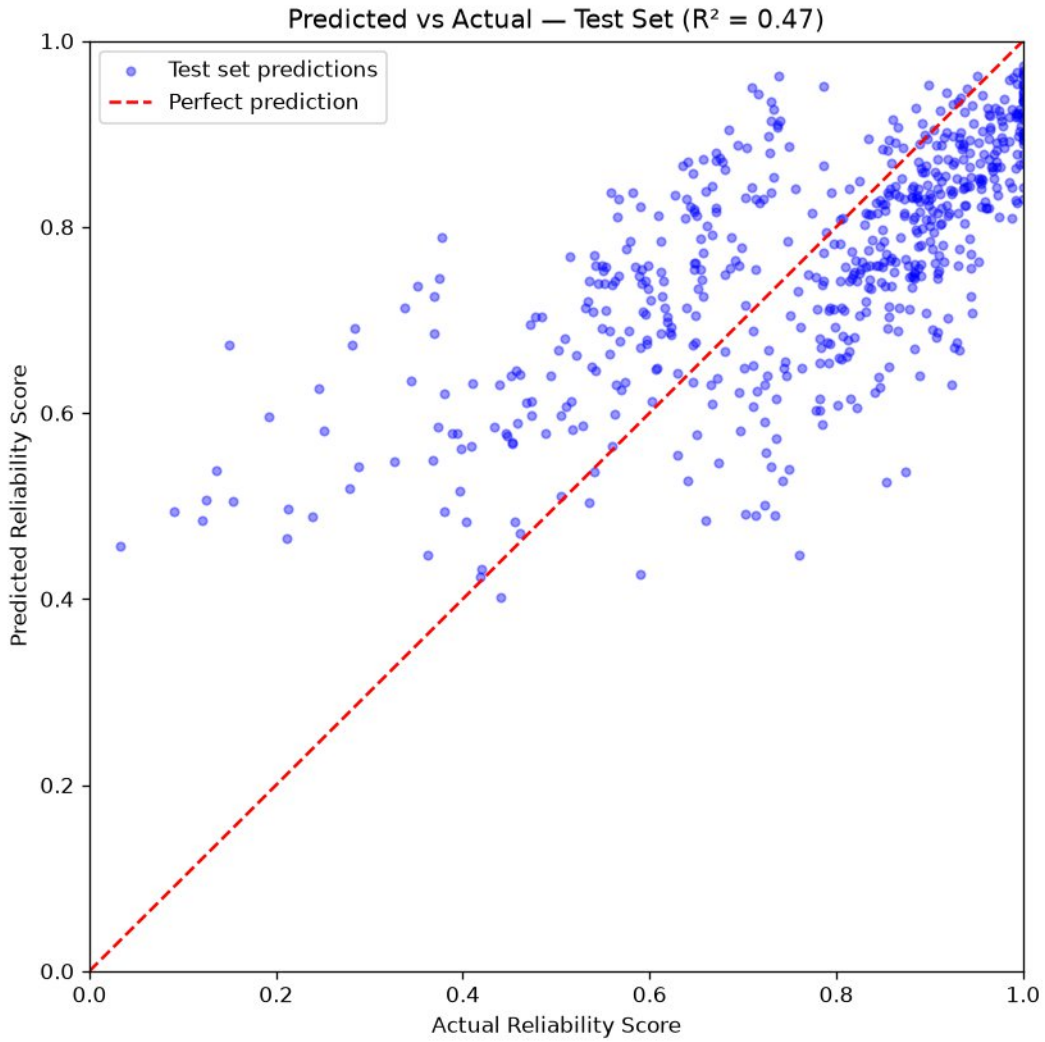



*Figure 5.3.4 Predicted against actual reliability score on the test set*

Figure 5.3.4 plots the predictions for the 600 test records against the actual scores. The points follow the diagonal, which confirms that the model has learned the trend, but they are spread widely around it. Records with a low actual score lie mostly above the line, so the model over-predicts the reliability of the least reliable bookings and compresses its predictions into a narrower range than the actual scores. An $R^2$ of 0.47 means that about half of the variance in the target is explained. The remainder reflects the random variation that the simulation adds to each outcome and any influences that are not among the five features.

### 5.3.5 Recommendation Service

The recommendation service in api.py combines the database, the feature engineering module and the saved model. When the service starts it loads the model and the reference tables into memory. A POST /recommend request is first validated, so that a time slot outside the four offered, a day type that contradicts the job date, or a date in the past or more than 60 days ahead is rejected with status 422. Providers of the requested service type without an availability record for the weekday and slot are removed, and a feature row is built for every remaining provider. The model scores each row, the providers are sorted by score, a rule-based explanation is attached, and the optional filters on rate, rating and verification are applied before the list is cut to the requested length. A separate endpoint lists new providers so that a provider with no history is not hidden (FR-08), and the booking endpoint recomputes the score on the server (FR-09).

Figure 5.3.5 shows the saved client interface for an electrician request in Kilimani during the evening rush on Monday 2 November 2026. The visible results rank Rose Ali first with a displayed reliability score of 78%, a distance of 8.41 km and an estimated travel time of 37.1 minutes, followed by Adhiambo Kamau at 77%, 9.75 km and 38.2 minutes. Each card displays a rate, rating and explanation. The explanation identifies heavy congestion, good road access and a strong historical completion rate. This interface capture is a separate observation from the earlier API example; the displayed values are rounded. The screenshot also appears in Figure A2.3; Figures A2.4 and A2.5 show the subsequent booking screens.

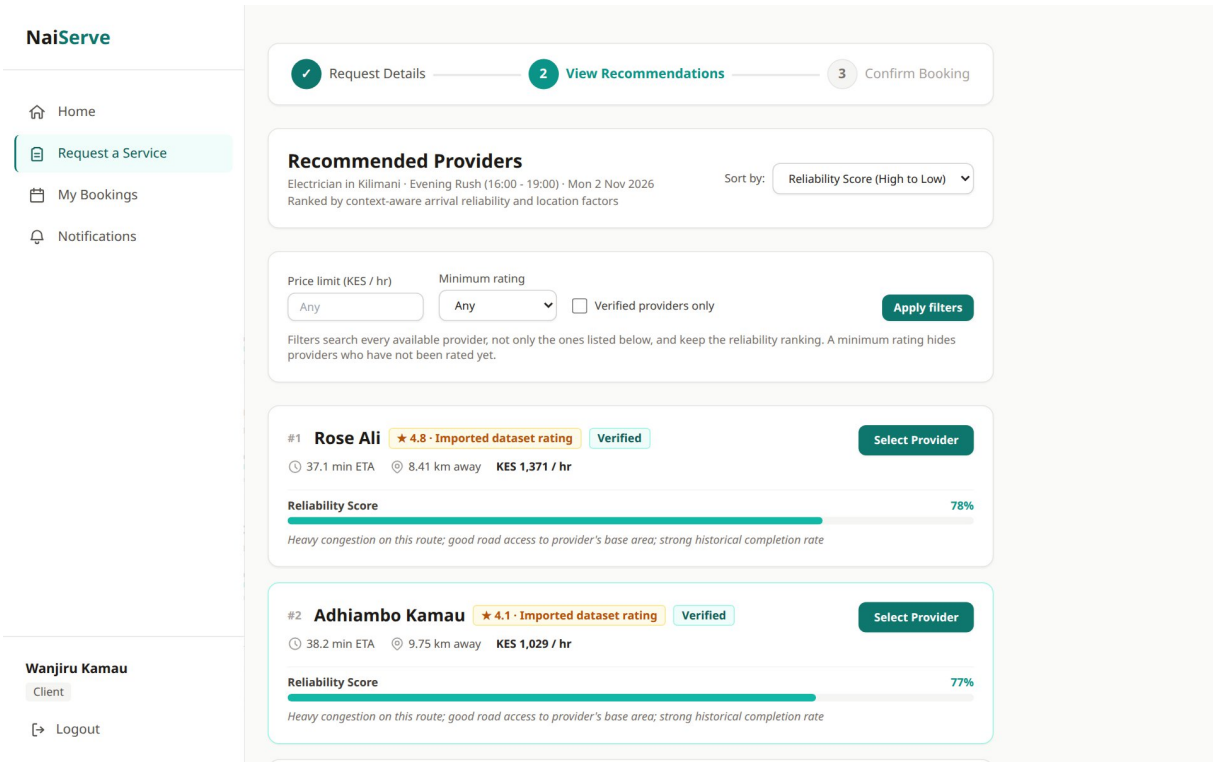



*Figure 5.3.5 Recommendation results in the working client interface*

## 5.4 Testing and Evaluation

This section reports whether the implemented system satisfies the requirements of Chapter 4. It follows the order of test strategy, test cases and defects, project-specific evaluation and interpretation. Every result was produced on the development workstation of Table 5.1, against the local PostGIS container.

### 5.4.1 Test Strategy and Acceptance Criteria

The purpose of testing was to show that each requirement of Chapter 4 that has a verification method is met by the implementation, and that the model satisfies NFR-01. The scope is the FastAPI service with its database, the model and the browser interface. Four levels were used because each answers a different question (Table 5.7). The test data are the synthetic reference data of Section 5.3.2 and the accounts and bookings that the tests create for themselves, so no data about a real person were used. The author ran the tests, and the completion criterion for the automated levels is that every test passes when the documented environment variables are set.

*Table 5.7 Test levels, tools and acceptance criteria*

| Level | Question answered | Tool and scope | Acceptance criterion |
|---|---|---|---|
| Unit | Do feature computation and input validation return the right values? | pytest 8.3.3; test_feature_engineering.py and the validation tests | Every test passes |
| Integration | Do the API, the database and the model work together, including error responses? | pytest with the FastAPI test client (httpx 0.27.2); 205 tests in 15 files; tests that need a live database run only with RUN_DB_TESTS=1 against an isolated PostGIS database | Every test passes |
| System | Can a client and a provider complete their flows in a browser? | Playwright 1.59.1; 20 tests in 12 files in tests/e2e | Every test passes on a run against the running stack |
| Model experiment | Does the model beat both baselines on unseen data? | Held-out 600-record test set (Section 5.3.4) | Lower RMSE and MAE than both baselines (NFR-01) |
| Performance | How long does a recommendation request take? | 200 sequential requests (Section 5.4.3) | A limit is fixed after benchmarking the deployed environment (NFR-05) |

### 5.4.2 Test Cases, Results, Defects and Retesting

Table 5.8 summarises the 205 automated backend tests by area and traces each area to the requirements it verifies. The test cases themselves are the functions in tests/ml-service, whose names state the scenario and whose assertions state the expected result, so another evaluator can repeat them with the command in Section 5.2.3. With RUN_DB_TESTS=1 and the database variables set, all 205 tests passed in 15.98 seconds with 7 warnings. The warnings that were displayed are a pandas FutureWarning raised when a newly registered provider is appended to the in-memory provider table, and they do not affect any result.

*Table 5.8 Automated backend tests by area*

| Area (test file) | Tests | Requirements verified | Result |
|---|---|---|---|
| Core endpoints (test_api_core) | 10 | FR-03, FR-05, FR-06, IR-05 | All passed |
| Context validation (test_api_context_validation, test_past_time_slots) | 10 | DR-04 | All passed |

| Area (test file) | Tests | Requirements verified | Result |
|---|---|---|---|
| Feature engineering (test_feature_engineering) | 9 | DR-02, DR-03, FR-05 | All passed |
| Registration and login (test_api_auth) | 8 | FR-01, FR-02, NFR-02 | All passed |
| Availability (test_api_availability) | 12 | FR-04, FR-12 | All passed |
| Result filters (test_api_filters) | 13 | FR-07 | All passed |
| New providers (test_cold_start) | 13 | FR-08, DR-06 | All passed |
| Bookings and status changes (test_api_bookings) | 48 | FR-09, FR-10, FR-14, IR-05 | All passed |
| Dated scheduling (test_api_scheduling) | 11 | FR-09, DR-04 | All passed |
| Rescheduling (test_api_rescheduling) | 34 | FR-11 | All passed |
| Reviews (test_api_reviews) | 12 | FR-13, DR-06 | All passed |
| Profile and notifications (test_api_profile_notifications) | 8 | FR-14, FR-15 | All passed |
| Exact location (test_exact_location) | 13 | FR-15, IR-04 | All passed |
| Administrator functions (test_api_admin) | 4 | FR-16, FR-17, NFR-03 | All passed |
| Total | 205 | | All passed |

Positive tests show that valid input is accepted. The negative and boundary cases in Table 5.9 matter more for correctness because they show that invalid input is refused with the status code that IR-05 prescribes. The two database cases were carried out by hand in SQL, as described in Section 5.3.1; the others are automated tests.

*Table 5.9 Selected negative and boundary test cases*

| ID | Requirement | Scenario | Expected result | Actual result and verdict |
|---|---|---|---|---|
| TC-01 | DR-04, IR-05 | POST /recommend with a time slot or day type that does not exist | Rejected with 422 | 422 returned. Pass |

| ID | Requirement | Scenario | Expected result | Actual result and verdict |
|---|---|---|---|---|
| TC-02 | DR-04 | POST /recommend for a morning slot that has already ended today, at the boundary minutes | Request rejected | Rejected at and after the boundary; current and future slots accepted. Pass |
| TC-03 | FR-01 | Register with an email address that already has an account | Rejected with 409 | 409 returned. Pass |
| TC-04 | FR-01 | Register a provider without the provider-specific fields | Rejected with 400 | 400 returned. Pass |
| TC-05 | FR-02 | Log in with a wrong password; log in with an unknown email | Both rejected with 401 | 401 in both cases. Pass |
| TC-06 | FR-12 | Save availability with invalid or missing slots; save for an unknown provider | 422 for invalid or missing slots; 404 for an unknown provider | As expected. Pass |
| TC-07 | FR-17, NFR-03 | A non-administrator requests the administrator provider list | Rejected with 403 | 403 returned. Pass |
| TC-08 | FR-17 | Request model retraining while a retraining job is already active | No second job is started | Single active job enforced. Pass |
| TC-09 | NFR-04 | Insert a user with role client and a provider identifier | Rejected by the database | Violation of chk_users_role_matches_fk. Pass |
| TC-10 | NFR-04, DR-01 | Insert a historical booking with a reliability score of 1.5 | Rejected by the database | Violation of the check constraint on arrival_reliability_score. Pass |

Table 5.10 records the defects found while the evidence for this chapter was collected, including a failed run, which is reported and not hidden because a later run passed.

*Table 5.10 Defects found, corrections and retests*

| ID | Defect | Cause | Correction and retest |
|---|---|---|---|
| D-01 | The first run of the backend suite gave 172 passes, 10 failures and 13 skips. | The database environment variables were not set, so the ten database-dependent tests could not connect. | The variables were set as documented in the README. The rerun passed 195 tests, and 205 after the administrator tests were added. |
| D-02 | The landing and registration pages described the recommendations as using “real” or “live” traffic. | The wording overstated the data: the system uses stored corridor and time-slot averages and has no live traffic feed. | The wording was changed to “typical traffic”. A search of the frontend source for both phrases found no match and the TypeScript check passed. |
| D-03 | The saved model verification note reported a test RMSE of 0.1419 and an MAE of 0.1162. | The note recorded an earlier run, and the current pinned pipeline gives slightly different values. | The note was updated to 0.1417 and 0.1160, the values reproduced with the same split and seed. |

### 5.4.3 Model, Performance and Interface Evaluation

Model evaluation. The model was compared with two baselines on the same 600 held-out records, and its variation was measured across five folds (Section 5.3.4). Retraining the model with the same data, split and seed reproduced the test RMSE of 0.1417, the MAE of 0.1160 and the $R^2$ of 0.4717, and the five shuffled folds gave a mean RMSE of 0.1414 with a standard deviation of 0.0020. The training RMSE of 0.1316 is 0.010 below the test RMSE, so the model fits the training records only slightly better than unseen ones. A breakdown of the errors by condition is given in Section 5.7.2.

Performance. NFR-05 asks for a response-time limit that is fixed by benchmarking. A benchmark was run on the development workstation by sending 200 sequential POST /recommend requests for an electrician in Kilimani, on a weekday evening rush, with the top five results, after one warm-up request. The client ran in the same process as the API, so network time is not included, and the database was the local PostGIS container. Figure 5.4.1 shows the new run captured on 6 November 2026: all 200 measured requests succeeded, with a median of 127.4 ms, a 95th percentile of 130.6 ms and a maximum of 190.4 ms. The hosted deployment on Render and Neon was not benchmarked, and no limit has yet been fixed; NFR-05 is therefore only partly evidenced.

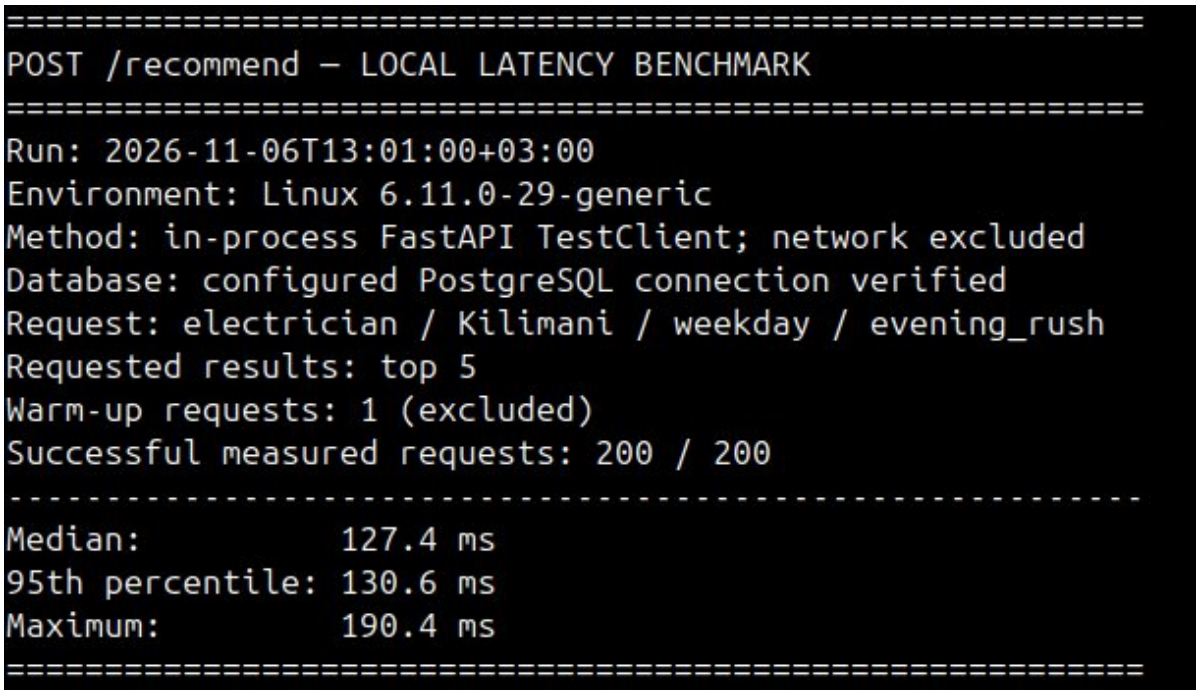



*Figure 5.4.1 Latency of POST /recommend on the development workstation (200 requests)*

Interface. The Next.js production build compiled, passed type checking and generated 21 routes in the documentation audit, and the TypeScript check passed again after the wording change in D-02. The Playwright suite contains 20 tests in 12 files that cover registration and login, the client and provider flows, scheduling, rescheduling, cancellation, reviews, exact location, result filters, new providers, error states and installability. The suite was not rerun for this chapter because every run creates accounts and bookings that are not removed, and the development data were kept. The stored report predates the administrator functions and this chapter, so no pass count for the browser tests is claimed here. Screenshots of the client, provider and administrator screens are in Appendix A2 (Figures A2.1 to A2.8).

Human-centred evaluation. No usability test with users, accessibility audit or security test such as the OWASP Web Security Testing Guide has been carried out. The informal usability check planned for the project (issue 15) is still open. NFR-06 is supported by the fact that every recommendation carries an explanation string (Section 5.3.5), but whether clients understand and trust it has not been measured.

### 5.4.4 Interpretation of the Test Results

The results show that the service behaves as specified for the cases that were tested, that invalid input is refused with the intended status codes, that the database protects its own integrity and

that the model result can be reproduced. They do not show that the interface is easy to use, that the service is fast under load or in the hosted environment, or that the system is secure, because those were not tested. A passing test demonstrates the absence of failure for the tested case only, and the suite was written by the author who wrote the code, so it is likely to share the author's assumptions.

## 5.5 Version Control and Implementation Evidence

The project repository is at https://github.com/Theedxctor/nairobi-service-recommender and is public, so the supervisor can read the code and the history. At the time of writing the history held 106 commits, 54 issues of which 41 were closed, and 43 pull requests of which 42 were merged. Issues were assigned to the milestones M1 to M8, which run from the ML pipeline to the final documentation and the marketplace follow-ups.

Once the workflow was adopted, every change followed one workflow: an issue, a branch named feat, fix, docs or refactor with a short description, commits in the conventional format with the issue number in the footer, and a pull request that was merged with a merge commit and not squashed, so that branch and pull-request structure stays visible in the history. The pull request was the point at which the author reviewed a change before approving it. Commits made before that point went directly to the main branch and the history was not rewritten, so the early part of the history does not follow the workflow. Table 5.11 lists the artefacts that make the work reproducible.

*Table 5.11 Reproducibility artefacts in the repository*

| **Artefact** | **Purpose** |
|---|---|
| README.md | Problem, setup, configuration, run and test instructions, and the current model results |
| database/schema.sql; database/migrations (002 to 007) | Full schema and the additive upgrades for databases created earlier |
| database/migrate_csv_to_postgres.py; database/seed_admin.py | Idempotent data load with a --reset option; the only way to create an administrator account |
| ml-service/requirements.txt; requirements-api.txt; frontend/package.json | Exact Python versions for development and for the deployed image; frontend dependencies |
| ml-service/finalize_model.py | Trains the pipeline with a fixed seed and writes ml-service/results/latest_model_metrics.json |

| Artefact | Purpose |
|---|---|
| ml-service/results/model_performance_report_2026-10-27.md | Training, validation and test results with the protocol used |
| render.yaml; docs/DEPLOYMENT.md | Configuration of the hosted API, database and frontend |
| tests/ml-service; tests/e2e | Automated backend tests and browser tests |

Secrets are kept out of the repository: environment files are excluded by .gitignore and the hosted credentials are held in the Render and Vercel dashboards. Three gaps remain. The repository has no licence file, no release tag marks the submitted version, and no deployed web address is recorded in it, so the hosted deployment was not independently verified for this report. The evidence in this chapter refers to the committed repository plus the administrator functions, which were in the working tree and not yet committed when it was collected; they should be committed and tagged before submission so that the report can be matched to an exact version.

## 5.6 Author Oversight and Use of AI-Assisted Tools

Claude Code, a coding assistant from Anthropic, was used during the implementation and testing of the system. It worked under written project rules that forbid fabricated data and claims without evidence. It assisted with refactoring backend and frontend code, running the tests and the model reproduction. The product itself contains no generative AI component, the recommendation model is a gradient-boosted regression trained on tabular data.

Its output was checked in four ways: figures in the report were traced to the commands that produced them (the test run, the model reproduction, the latency benchmark and SQL queries on the database); code changes were reviewed through pull requests before merging; the report was compared against the source files in a documentation audit; and statements that could not be supported were removed or reworded, as in defects D-02 and D-03. The author made the final decisions and takes full responsibility for the submitted work.

## 5.7 Evaluation Results, Discussion and Limitations

This section organises the evidence by research objective, discusses why the model behaved as it did and where it fails, reports the status of the requirements, and states the limitations. The conclusions, recommendations and future work are in Chapter 6.

### 5.7.1 Results by Objective

*Table 5.12 Results by research objective*

| Objective | Output and acceptance criterion | Observed result | What the result permits |
|---|---|---|---|
| i. Identify the factors that influence reliability | Five features, one per pillar, selected so that they match the larger candidate set (5.3.3); feature importance reported | Congestion 51.5%, completion rate 16.5%, time slot 11.9%, road quality 10.3%, distance 9.8% (Table 5.6) | In this synthetic dataset congestion matters most and distance least. The ranking is not evidence of causes in Nairobi. |
| ii. Review existing platforms and identify gaps | Chapter 2 gap analysis of Mama Fua, Fundis and Kazi Konekt | None of the reviewed platforms was found to use traffic, time-of-day or road-quality context | A gap among the reviewed platforms. It is not a claim about every platform in Kenya. |
| iii. Develop a model that predicts arrival reliability | Pipeline served by the API (FR-03 to FR-06, IR-03); all related tests pass | Working service with 205 passing backend tests; example ranking in Figure 5.3.5 | A working prototype that scores providers on simulated data. It predicts a synthetic reliability score and not a real arrival time. |
| iv. Evaluate against distance-based and rating-based approaches | Lower RMSE and MAE than both baselines on the same test set (NFR-01) | RMSE 0.1417 against 0.1872 and 0.1936; MAE 0.1160 against 0.1514 and 0.1585; $R^2$ 0.4717 against 0.0776 and 0.0141 | The model is better on this dataset. The result depends on how the simulator was built. |
| General objective: a working system for clients and providers | Requirements FR-01 to FR-17 implemented and tested | All 17 functional requirements have an implementation. Administrator verification is a profile and contact review and not a check of government identity. | A demonstrable prototype. It is not production-ready because authentication is not verified on the server. |

The fourth research question asks about improvement in recommendation accuracy. The study measured the error of the predicted reliability score and not the quality of the ranked lists, because there are no real bookings from which a correct ranking could be known. The result therefore shows that the score is closer to the simulated outcome than the baselines are, and it does not show that clients receive better providers.

### 5.7.2 Discussion and Error Analysis

The model beat both baselines by a wide margin because, in the simulated data, reliability depends mainly on congestion, provider history and their interaction with the time of day, and a

straight line through distance or rating cannot represent that. The comparison should be read with care for two reasons. The baselines are linear regressions on one variable, so the gap combines the effect of the richer feature set with the effect of a non-linear model, and a linear model on the five features, or an XGBoost model on distance alone, was not run to separate the two. The untuned XGBoost model (RMSE 0.1583) lies between the baselines and the tuned model, so tuning accounted for a further 10.5% reduction in RMSE.

Table 5.13 shows where the errors are on the 600 test records. They are largest where the system is needed most: in the evening rush, under heavy congestion and on poor roads. The clearest pattern is that the model is least accurate for the least reliable bookings. For the 60 records whose actual score is below 0.5, the RMSE is 0.2589 and the model over-predicts by 0.226 on average, while it under-predicts the most reliable bookings by 0.086. This is the compression towards the mean that was visible in Figure 5.3.4, and it means that a risky booking can be ranked higher than it deserves.

*Table 5.13 Test-set error by condition (actual minus predicted score)*

| **Condition** | **Records** | **RMSE** | **Mean error** |
|---|---|---|---|
| Time slot: mid_morning | 109 | 0.1178 | +0.0204 |
| Time slot: midday | 164 | 0.1281 | +0.0106 |
| Time slot: early_afternoon | 123 | 0.1326 | +0.0106 |
| Time slot: morning_rush | 109 | 0.1591 | −0.0127 |
| Time slot: evening_rush | 95 | 0.1752 | −0.0182 |
| Road quality: good | 225 | 0.1247 | +0.0104 |
| Road quality: moderate | 179 | 0.1418 | −0.0007 |
| Road quality: poor | 196 | 0.1589 | −0.0003 |
| Congestion multiplier ≤ 1.5 | 112 | 0.1110 | +0.0178 |
| Congestion multiplier 1.5 to 2.5 | 313 | 0.1312 | +0.0098 |
| Congestion multiplier > 2.5 | 175 | 0.1737 | −0.0167 |
| Actual score below 0.5 | 60 | 0.2589 | −0.2260 |
| Actual score 0.5 to 0.8 | 208 | 0.1425 | −0.0620 |
| Actual score above 0.8 | 332 | 0.1069 | +0.0861 |

Two further observations follow from the results. First, the three scores in Figure 5.3.5 lie within 0.024 of each other, which is far smaller than the RMSE of 0.14, so the order of providers whose scores are this close is not reliable and clients should be shown the score and the explanation and not only the rank. Second, no weekend_day booking appears in the training data, so the scores for that slot are extrapolations. The explanation text names the bands of the factors and does not

measure their individual contributions, which limits what NFR-06 achieves.

### 5.7.3 Requirement Coverage

All functional requirements FR-01 to FR-17 have an implementation and an automated or inspection-based verification method, as shown in Table 4.1 and Table 5.8. Table 5.14 gives the status of the non-functional requirements, where the evidence is uneven.

*Table 5.14 Status of the non-functional requirements*

| ID | Status | Evidence |
|---|---|---|
| NFR-01 | Met | Lower RMSE and MAE than both baselines on the 600-record test set (Table 5.5) |
| NFR-02 | Met in the tested paths | Only bcrypt hashes are stored (password_hash column); the eight authentication tests pass |
| NFR-03 | Partly met | Role checks guard the interface and administrator endpoints return 403 to other roles, but identity is not verified on the server (issue 84) |
| NFR-04 | Met | Both constraint-violation checks were rejected by the database (TC-09, TC-10) |
| NFR-05 | Partly evidenced | Median 127.4 ms on the workstation (Figure 5.4.1); no limit fixed; hosted environment not benchmarked |
| NFR-06 | Partly met | Every recommendation carries an explanation; no user evaluation of understanding or trust |
| NFR-07 | Not fully verified | Browser tests for responsive layout and installability exist but were not rerun; no device matrix was tested |
| NFR-08 | Met | Clean rebuild from an empty database; pinned dependencies; the model result reproduced on repeated runs |
| NFR-09 | Met | 205 backend tests pass with the documented environment |
| NFR-10 | Not re-verified | Dashboards were inspected earlier, before the administrator pages were implemented, and were not inspected again for this chapter |

### 5.7.4 Limitations

Table 5.15 lists the limitations that affect how far the findings can be trusted. None has been turned into a stronger claim.

*Table 5.15 Limitations of the study*

| Limitation | Effect on the findings |
|---|---|
| Synthetic data | All 3,000 training records were simulated. The results describe the behaviour of the system on simulated Nairobi data and say nothing about real providers, and the feature importance reflects relationships that were built into the simulator. |

| Limitation | Effect on the findings |
|---|---|
| Evaluation design | The split is random across bookings and not grouped by provider, so the test measures new bookings of known providers and not new providers. Only one split was held out, and the baselines are single-variable linear models. |
| Stored traffic patterns | Congestion comes from stored corridor and time-slot averages. The system has no live traffic feed and cannot react to an incident on a given day. |
| Target and outcome | The model predicts a synthetic reliability score. It does not predict arrival time, and no study measured missed appointments or client satisfaction. |
| No user study | Usability, accessibility and the clarity of the explanations were not tested with users. |
| Security | Authentication is not verified on the server, so the prototype must not be used with real accounts. Administrator verification reviews profile and contact details and does not check a government identity document. No deletion or retention policy exists (DR-08). |
| Environment | Performance was measured in one environment with sequential requests and no load test, and the hosted deployment was not independently verified. |

# References


Adomavicius, G., Mobasher, B., Ricci, F., & Tuzhilin, A. (2011). Context-aware recommender systems. *AI Magazine*, *32*(3), 67–80.

Bast, H., Delling, D., Goldberg, A. V., Müller-Hannemann, M., Pajor, T., Sanders, P., Wagner, D., & Werneck, R. F. (2016). Route planning in transportation networks. In L. Kliemann & P. Sanders (Eds.), *Algorithm engineering* (pp. 19–80). Springer. https://doi.org/10.1007/978-3-319-49487-6_2

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Daily Nation. (2025, October 27). *Gridlocked capital: Mapping Nairobi's worst traffic hotspots and timings*. Nation Media Group. https://nation.africa/kenya/counties/nairobi/

Fundis App. (2023). *Fundis: Find skilled workers near you* [Mobile application]. Fundilink Technologies.

Haer, T., et al. (2025). *Towards modeling road access deprivation in Sub-Saharan Africa based on a new accessibility metric and road quality* (arXiv:2512.02190). arXiv. https://arxiv.org/abs/2512.02190

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Huang, H., Pouls, M., Meyer, A., & Pauly, M. (2020). Travel time prediction using tree-based ensembles. In E. Lalla-Ruiz (Ed.), *Computational logistics: 11th International Conference, ICCL 2020, Enschede, The Netherlands, September 28–30, 2020, Proceedings* (pp. 412–427). Springer. https://doi.org/10.1007/978-3-030-59747-4_27

ISO/IEC. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Product quality model*. International Organization for Standardization.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*. Institute of Electrical and Electronics Engineers.

Japan International Cooperation Agency. (2006). *Nairobi urban transport master plan study*. Ministry of Roads and Public Works, Kenya. https://openjicareport.jica.go.jp/pdf/11823093_02.pdf

Kazi Connect. (2023). *Kazi Konekt: On-demand worker dispatch platform* [Mobile application]. Kazi Technologies.

Kenya National Bureau of Statistics. (2019). *Kenya population and housing census volume I: Population by county and sub-county*. Kenya National Bureau of Statistics. https://www.knbs.or.ke

Kenya Roads Board. (2023). *Kenya road inventory and conditions survey 2023*. Government of Kenya. https://maps.krb.go.ke

Mama Fua. (2023). *Mama Fua: Professional cleaning services* [Mobile application]. Mama Fua Ltd.

tono, N., Wright, J. A., Mutunga, M., Mutembei, H., & Thumbi, S. M. (2022). Impact of traffic congestion on spatial access to healthcare services in Nairobi. *Frontiers in Health Services, 2*, Article 788173. https://doi.org/10.3389/frhs.2022.788173

Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender systems: Introduction and challenges. In F. Ricci, L. Rokach, & B. Shapira (Eds.), *Recommender systems handbook* (2nd ed., pp. 1–34). Springer. https://doi.org/10.1007/978-1-4899-7637-6_1

Shah, U. (2021). *Nairobi's growing nucleus and deteriorating speeds*. Uber Movement Blog. https://medium.com/uber-movement/nairobis-growing-nucleus

TomTom International BV. (2025). *TomTom traffic index 2025: Nairobi*. https://www.tomtom.com/traffic-index/city/nairobi

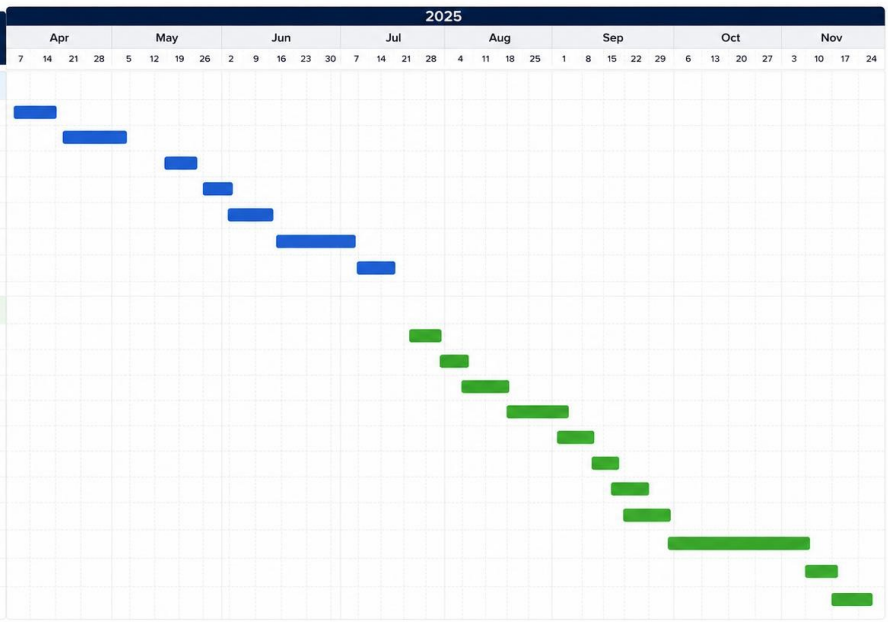
Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining* (Vol. 1, pp. 29–39).

# Appendices

## Appendix A1: Time Schedule

### PROJECT TIMELINE (GANTT CHART)

| Task | Start | End |
|---|---|---|
| **SEMESTER 1 (07/04/25 – 18/07/25)** | | |
| Project conceptualization and concept note | 07/04/25 | 18/04/25 |
| Supervisor allocation and initial meeting | 21/04/25 | 02/05/25 |
| Proposal writing: Chapter 1 | 13/05/25 | 19/05/25 |
| Proposal writing: Chapter 2 | 23/05/25 | 28/05/25 |
| Proposal writing: Chapter 3 | 29/05/25 | 09/06/25 |
| Proposal examination | 16/06/25 | 04/07/25 |
| Proposal defense | 14/07/25 | 18/07/25 |
| **SEMESTER 2 (18/08/25 – 28/11/25)** | | |
| Set up environment and tools | 18/08/25 | 22/08/25 |
| Data preparation and cleaning | 25/08/25 | 29/08/25 |
| Model design | 29/08/25 | 08/09/25 |
| Train and evaluate the model | 08/09/25 | 19/09/25 |
| Build frontend UI | 22/09/25 | 26/09/25 |
| Integrate model with UI | 29/09/25 | 03/10/25 |
| Testing and bug fixing | 03/10/25 | 10/10/25 |
| Documentation writing | 06/10/25 | 17/10/25 |
| Finalize system and documentation | 20/10/25 | 17/11/25 |
| Final defense preparation | 17/11/25 | 21/11/25 |
| Final Defense | 24/11/25 | 28/11/25 |



**Appendix A2: System Screenshots**

This appendix shows the main screens of the system as it was built. The screenshots were taken from the running prototype with test accounts and the local database. The client screens follow one request: an electrician in Kilimani for the evening of Monday 2 November 2026.ws the main screens of the system as it was built. The screenshots were taken from the running prototype with test accounts and the local database. The client screens follow one request: an electrician in Kilimani for the evening of Monday 2 November 2026.

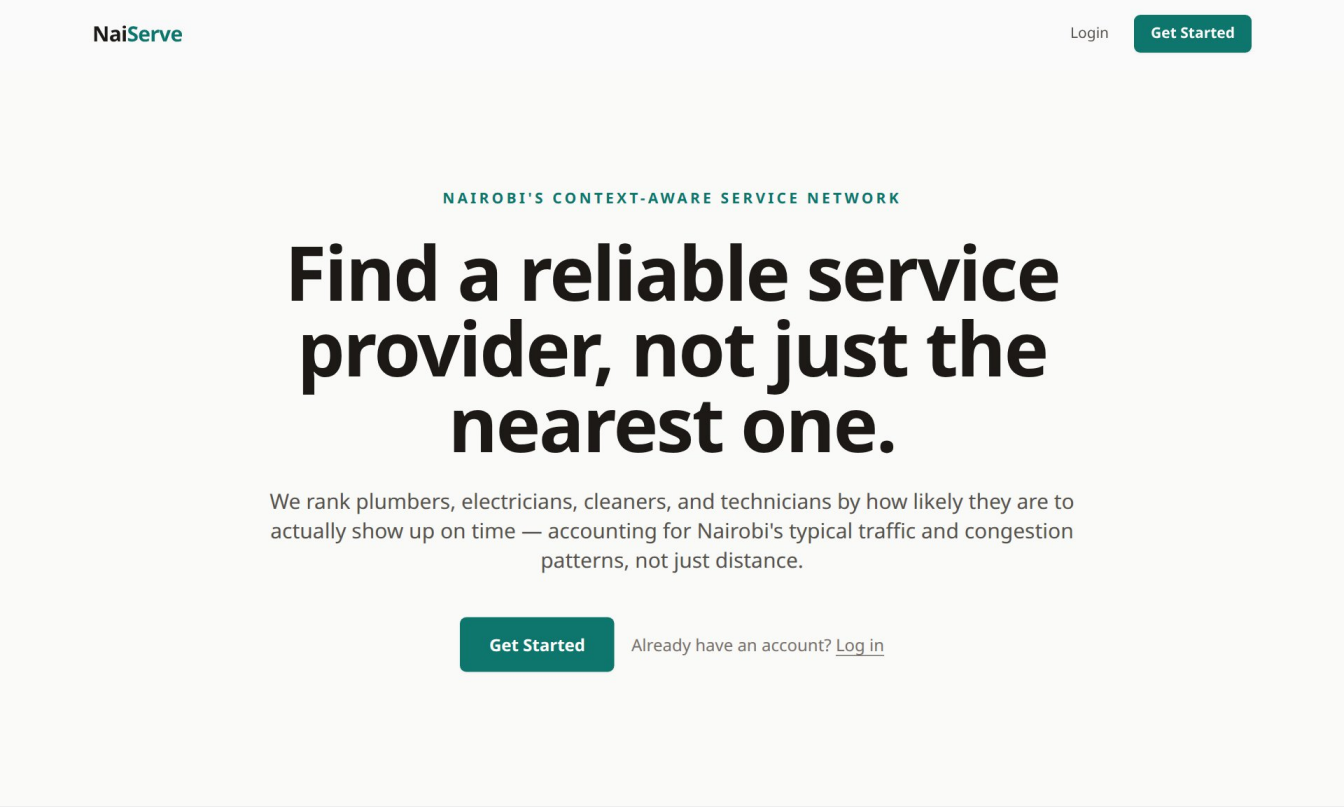



*Figure A2.1 Landing page*

Figure A2.1 shows the landing page, which explains what NaiServe does and takes the client to the request form.

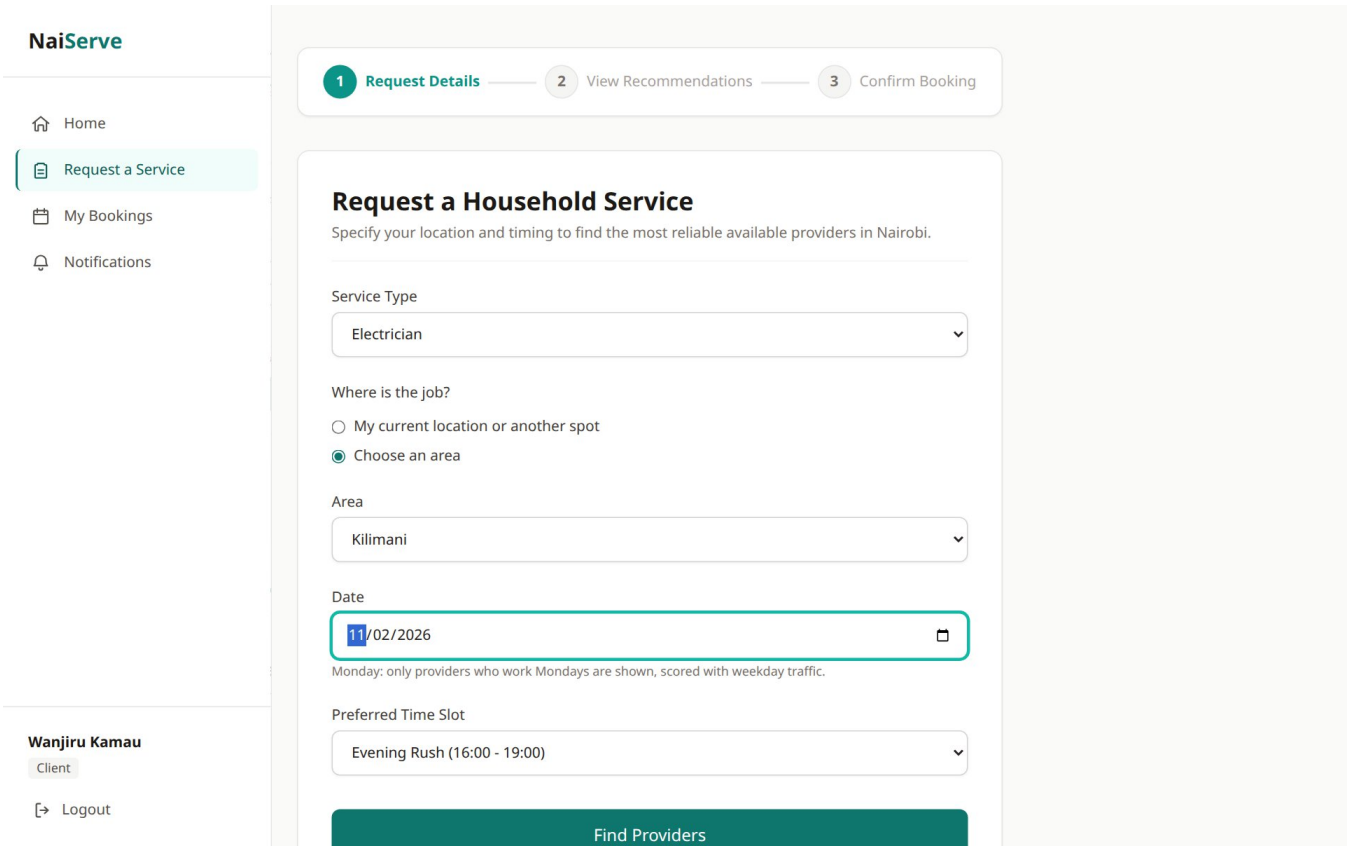



*Figure A2.2 Service request form*

In Figure A2.2 the client picks a service, a location, a date and a time slot. Time slots that have already ended cannot be chosen.

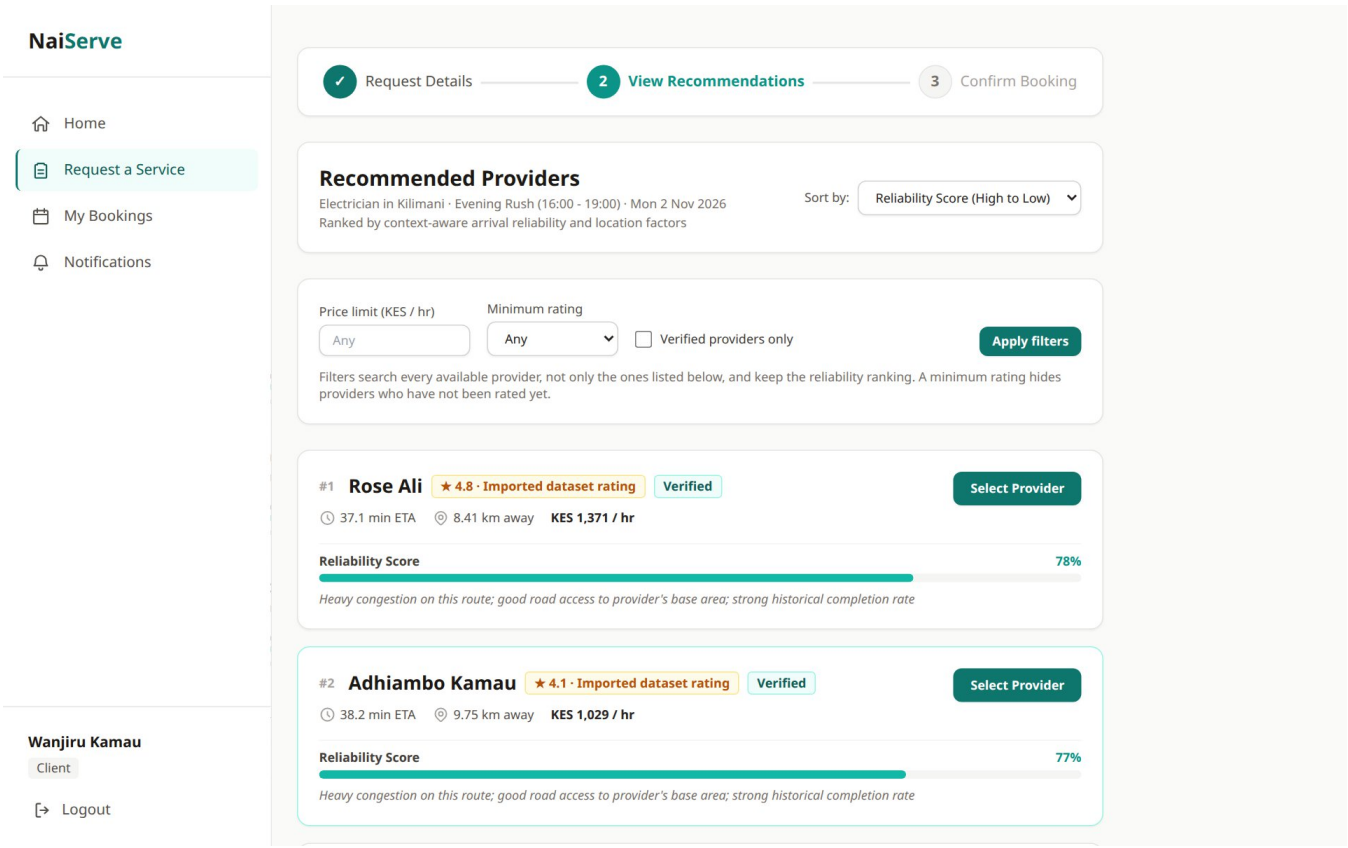



*Figure A2.3 Ranked provider results*

Figure A2.3 lists the providers from the highest to the lowest predicted reliability. Each card shows the travel time, distance, hourly rate and a short reason for the score.

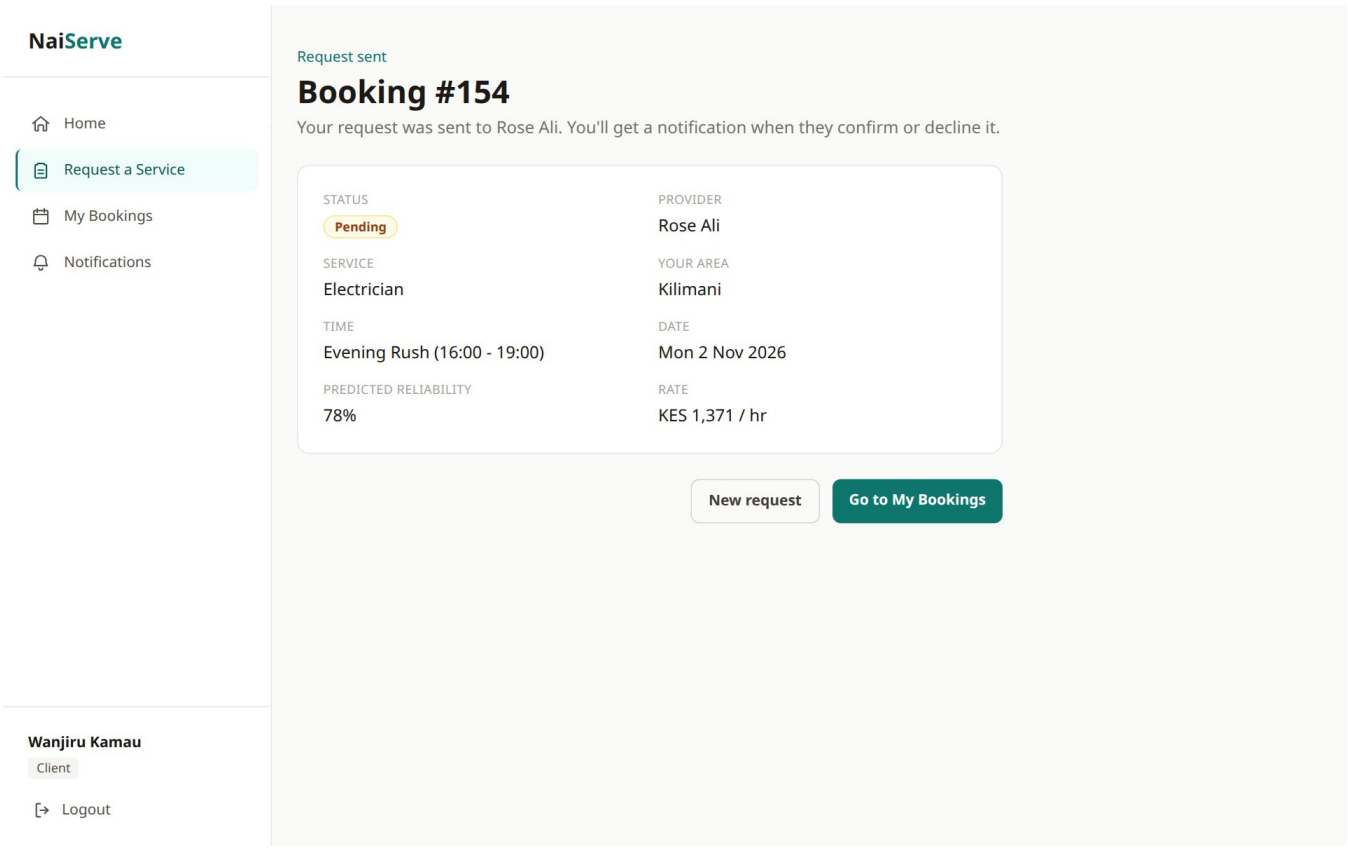



*Figure A2.4 Booking request sent*

When the client sends the request, the booking is saved as Pending and the provider is notified (Figure A2.4).

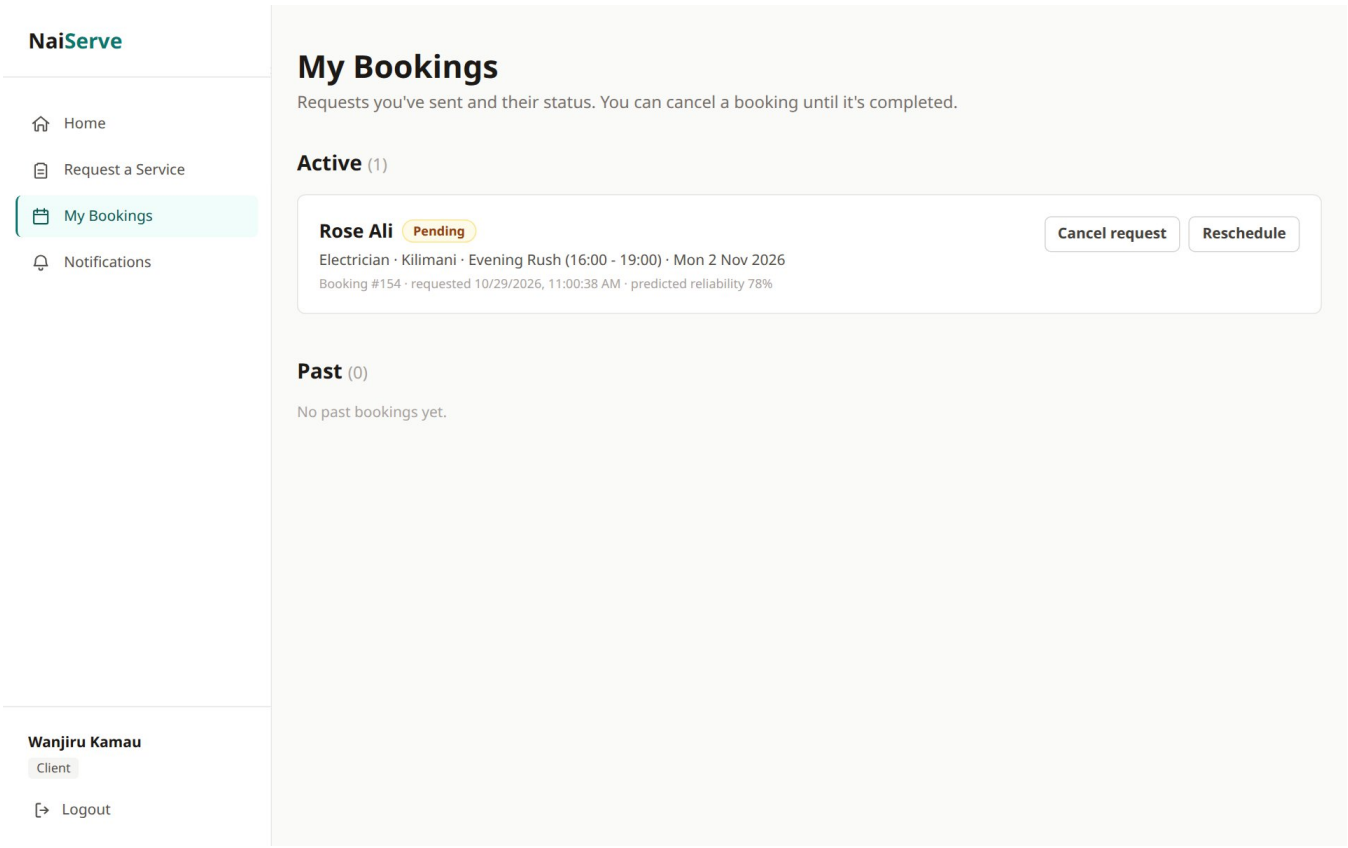



*Figure A2.5 Client booking list*

Figure A2.5 shows the client's bookings and their status.

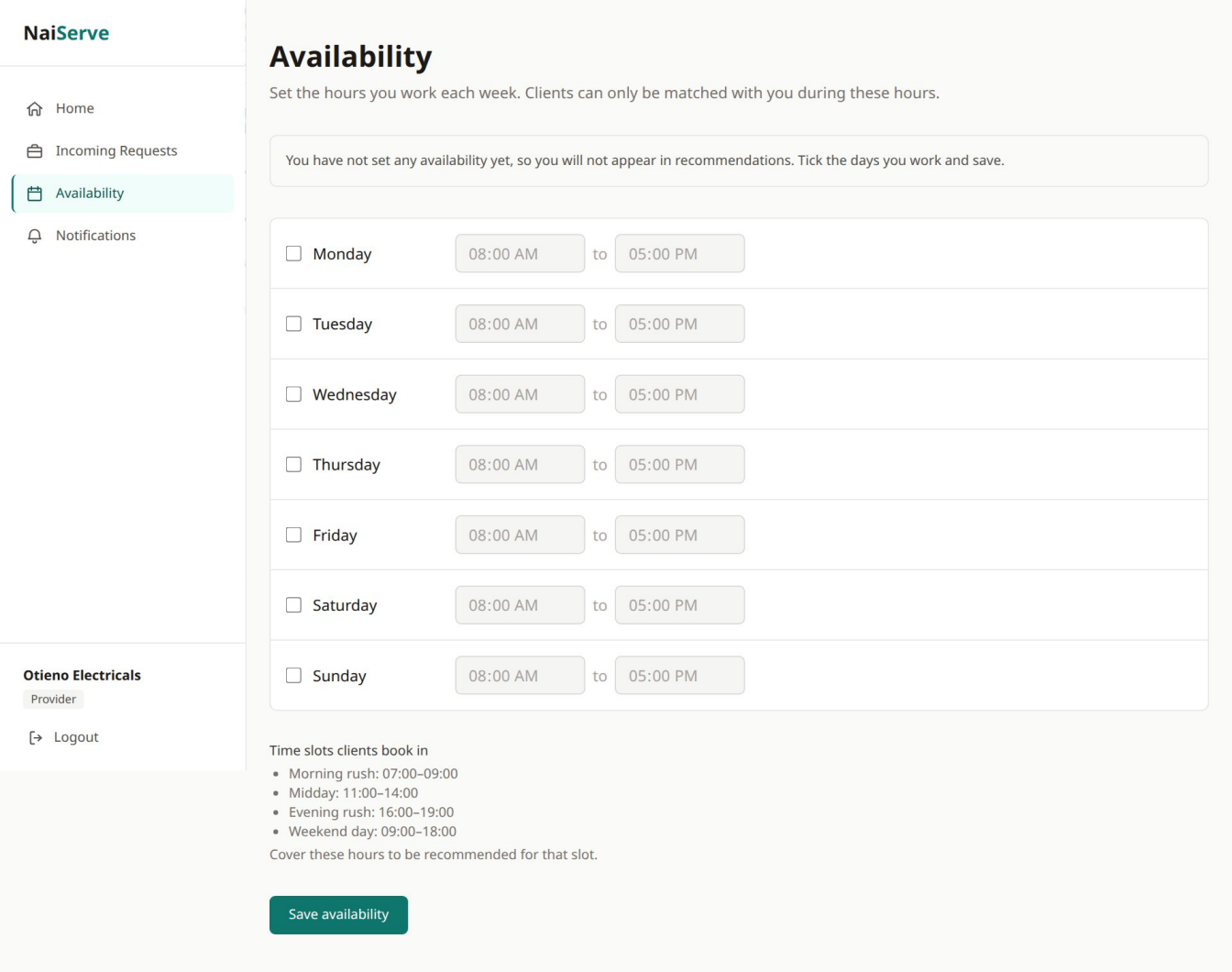



*Figure A2.6 Provider availability*

In Figure A2.6 a provider sets the days and time slots they work. Only providers who are available at the requested time are ranked.

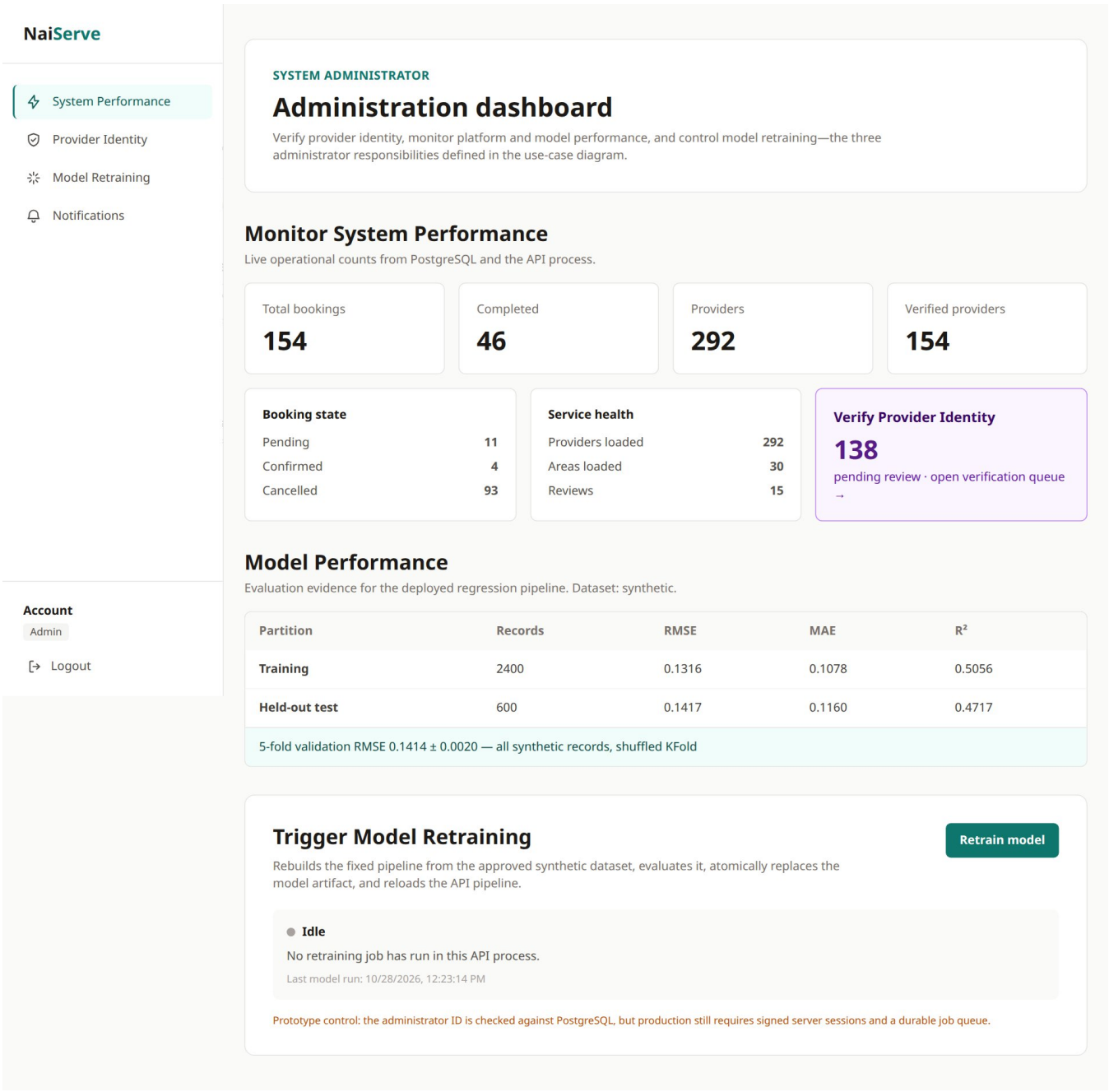



*Figure A2.7 Administrator dashboard*

Figure A2.7 shows the administrator dashboard, with booking and provider counts, the test results of the model and the button for retraining it.

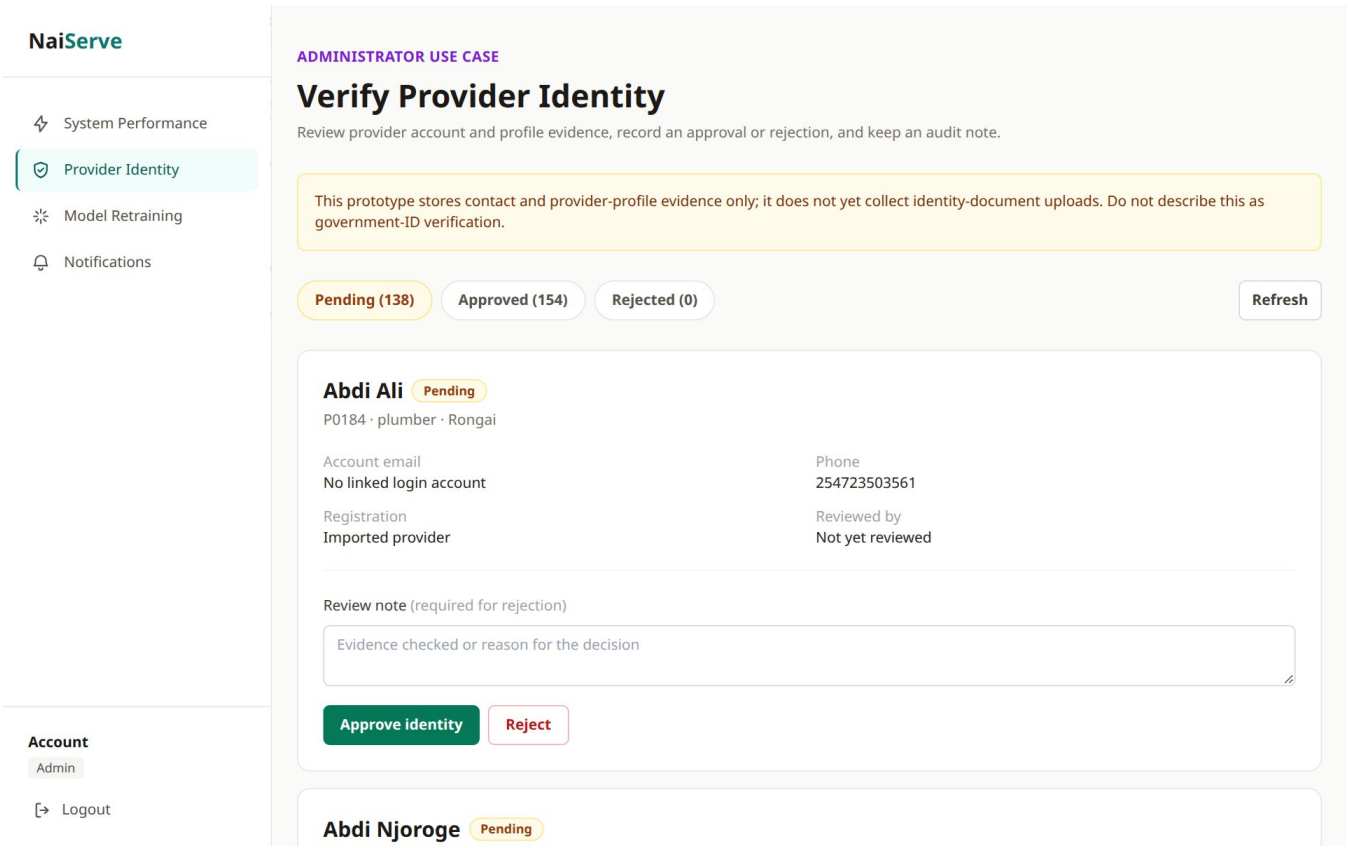



*Figure A2.8 Provider identity verification*

In Figure A2.8 the administrator approves or rejects providers waiting for verification and can add a note.